

ABAQUS 2025 RELEASE NOTES

Contents

Abaqus Release Notes.....	2
Introduction & Spatial Modeling	3
Abaqus/CAE.....	4
Analysis	8
Benchmarks	29
Constraints	30
Elements	32
Example Problems	35
Execution	38
Getting Started with Abaqus/CAE.....	40
GUI Toolkit	41
Interactions	42
Keywords	51
Materials	61
Output	72
Prescribed Conditions	79
Scripting	82
Scripting Reference	83
Theory	87
User Subroutines	88
Verification	90

Trademarks and Legal Notices

Trademarks

Abaqus, **3DEXPERIENCE**[®], the 3DS logo, the Compass icon, IFWE, 3DEXCITE, 3DVIA, BIOVIA, CATIA, CENTRIC PLM, DELMIA, ENOVIA, GEOVIA, MEDIDATA, NETVIBES, OUTSCALE, SIMULIA and SOLIDWORKS are commercial trademarks or registered trademarks of Dassault Systèmes, a European company (Societas Europaea) incorporated under French law, and registered with the Versailles trade and companies registry under number 322 306 440, or its subsidiaries in the United States and/or other countries. All other trademarks are owned by their respective owners. Use of any Dassault Systèmes or its subsidiaries trademarks is subject to their express written approval.

Legal Notices

Abaqus and this documentation may be used or reproduced only in accordance with the terms of the software license agreement signed by the customer, or, absent such agreement, the then current software license agreement to which the documentation relates.

This documentation and the software described in this documentation are subject to change without prior notice.

Dassault Systèmes or its Affiliates shall not be responsible for the consequences of any errors or omissions that may appear in this documentation.

© Dassault Systèmes Americas Corp., 2024.

For a full list of the third-party software contained in this release, please go to the Legal Notices in the Abaqus 2025 HTML documentation, which can be obtained from a documentation installation, or in the SIMULIA Established Products 2025 Program Directory, which is available on www.3ds.com.

Abaqus Release Notes

The Abaqus Release Notes contain brief descriptions of the new features available in the Abaqus product line.

Documentation updates after the GA (General Availability; the first release on a new level) release are delivered in FD (Functional Delivery) releases. If new functionality is added in an FD release with no documentation delivery, the information is included in the related Program Directory.

This guide is a part of the Abaqus[®] documentation collection, which describes all the capabilities of the Abaqus finite element analysis technology used in SIMULIA[®] applications.

Introduction & Spatial Modeling

This page describes recent changes in Introduction & Spatial Modeling.

2024 FD01

Generating a Node Set from a Surface

You can specify the name of a previously defined surface, in which case the nodes included in the surface will be assigned to the specified node set. This method can be used only to define sorted node sets.

Benefits: This surface-based node set feature makes it easier to create a node set that includes all the nodes on a predefined surface.

For more information, see *About Surfaces* and *Generating a Node Set from a Surface*.

Abaqus/CAE

This page describes recent changes in Abaqus/CAE.

2025 GA

Wear Modeling in Abaqus/CAE

Abaqus/CAE now supports the wear surface property and its assignment in general contact interactions.

Benefits: The support allows greater flexibility for modeling wear phenomena in Abaqus/CAE without having to use the keyword editor.

For more information, see *Specify wear surface property assignments* and *Defining a wear interaction property*.

Rotordynamic Loads in Abaqus/CAE

Abaqus/CAE now supports rotordynamic loads as a rotational body force type in Abaqus/CAE.

Benefits: This support allows greater flexibility to define a rotordynamic load as a rotational body force in Abaqus/CAE without having to use the keyword editor.

For more information, see *Defining a rotational body force*.

Step Dependence for Fluid Inflator and Fluid Exchange

Abaqus/CAE now supports the activation of fluid inflators and fluid exchange in steps.

Benefits: This support allows greater flexibility for modeling the inflator and cavity exchange in Abaqus/CAE without having to use the keyword editor.

Earlier releases of Abaqus/CAE supported automatic activation in Abaqus/Explicit steps, offering no flexibility to the user.

For more information, see *Defining a fluid inflator activation interaction* and *Defining a fluid exchange activation interaction*.

Modeling Cracks and Seams

Abaqus/CAE now supports the creation of a seam on a dependent part instance.

Benefits: Seam creation on a dependent part instance allows greater flexibility when modeling seams for fracture simulation in Abaqus/CAE.

Earlier releases of Abaqus/CAE supported seam creation only for independent instances.

For more information, see *The Special menu in the Interaction module*.

2024 FD04

Definition of Plastic Response

Abaqus/CAE now supports the definition of the plastic response that is used to compute Neuber and Glinka plasticity corrections.

Benefits: You can now natively use plasticity corrections in Abaqus/CAE to quickly estimate localized plastic strains.

For more information, see **PLASTICITY CORRECTION*.

2024 FD03

State Space Solver for Transient Modal Dynamic Analysis

The new state space solver for transient modal dynamic analysis of systems with nondiagonal modal damping operators is now the default solver in Abaqus/CAE.

Benefits: The state space solver is unconditionally stable and offers improved performance for models with many time increments and a moderate number of modes.

In earlier releases, the conventional solver was the default for transient modal dynamic analysis in Abaqus/CAE. Now, you must use the keywords editor to access the conventional solver in Abaqus/CAE.

For more information, see *Transient Modal Dynamic Analysis*.

Job Execution Using 3DEXPERIENCE On-Cloud Licensing

When you launch Abaqus/CAE from the 3DEXPERIENCE Platform Connector for Abaqus/CAE, you can now specify that jobs should execute using the 3DEXPERIENCE Platform license server instead of a locally installed license server.

Benefits: You can take advantage of available cloud SIM Unit licenses to execute an Abaqus analysis locally.

For more information, see *Setting the license type for the SimUnit license model*.

2024 GA

Python 3

Abaqus Python and Abaqus/CAE are now based on Python 3.10 instead of Python 2.7.

Benefits: This update brings Abaqus Python into conformance with a version of Python that is supported in the open-source community.

For more information, see <https://go.3ds.com/AbaqusPython>.

Render Styles in Plots for Continuum Particle Elements

Abaqus/CAE now supports visualization of element particle edges with wireframe, hidden, filled, and shaded render styles (**View > ODB Display Options**).

Benefits: The ability to assign different render styles makes it easier to distinguish between continuum particle elements.

For more information, see *Output, Output, and Controlling the display of model entities*.

Step-Dependent General Contact

Abaqus/CAE now supports the definition of general contact in Abaqus/Standard in any analysis step or in the initial step.

Benefits: Defining general contact in analysis steps enables definition of more complex contact behaviors in Abaqus/Standard simulations.

For more information, see *Defining general contact* and *About General Contact in Abaqus/Standard*.

Contact Mass Scaling

Abaqus/CAE now supports the definition of contact mass scaling in Abaqus/Explicit.

Benefits: Contact mass scaling can reduce the number of increments required for Abaqus/Explicit simulations that use nondefault penalty stiffness.

You can apply mass scaling to the contact surfaces or elements involved in the contact definition. Contact mass scaling reduces the impact of higher-than-default contact penalty stiffness on the global stable time increment in an Abaqus/Explicit analysis by increasing the mass associated with the contact surfaces.

For more information, see *Defining contact mass scaling* and **CONTACT MASS SCALING*.

Channel and Hat Section Profiles

You can now define channel and hat section profiles in the Property module.

Benefits: The addition of channel and hat profiles simplifies the specification of these common beam cross-sections in Abaqus/CAE.

For more information, see *Defining a channel profile*, *Defining a hat profile*, *ChannelProfile object*, *HatProfile object*, and *Beam cross-section analysis*.

Beam Section Offset for *BEAM SECTION

Abaqus/CAE now supports the definition of a beam section offset for beam sections integrated during the analysis (*BEAM SECTION).

Benefits: Beam offsets allow greater flexibility when modeling with beam elements in Abaqus/CAE. Earlier releases of Abaqus/CAE supported beam section offsets only for **BEAM GENERAL SECTION*.

For more information, see **BEAM SECTION, Using a Beam Section Integrated during the Analysis to Define the Section Behavior*, and *BeamSection object*.

Rotational Couplings

You can now define a rotational coupling type in discrete fasteners and coupling constraints in Abaqus/CAE.

Benefits: Defining the rotational coupling type gives you greater control over the behavior of distributing couplings involving cloud nodes with rotational degrees of freedom.

For more information, see **DISTRIBUTING*, *Creating discrete fasteners*, and *Defining coupling constraints*.

Using the Linearized Contact Capability to Solve for Contact Status and Contact Stresses

Abaqus/CAE now supports the activation of the linearized contact capability to solve for contact status and contact stresses within a static perturbation step with small-sliding, frictionless contact.

Benefits: The Linear Complementarity Problem (LCP) solution technique provides a faster solution for certain classes of contact problem.

For more information, see *Configuring a static, linear perturbation procedure* and **SOLUTION TECHNIQUE*.

Analysis

This page describes recent changes in Abaqus Analysis.

2025 GA

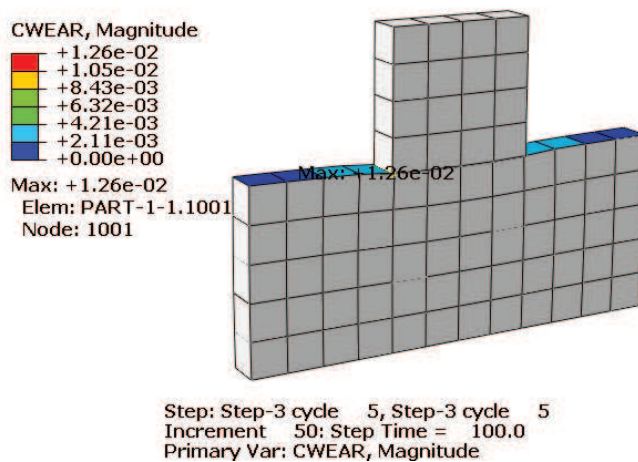
Step Cycling

The step cycling capability provides a simple way to repeat a step or a series of steps over a prescribed number of cycles.

Benefits: You can define cyclic repetition of a sequence of steps using the new step cycling capability instead of explicitly defining steps for every cycle, which provides simpler model setup, better performance, and the ability to end step cycling based on the analysis solution.

There are situations where the analysis requires the repetition of a step, or a series of steps, over a specified number of cycles. Examples of such workflows include aging in batteries, brake wear analysis, and fatigue. In earlier releases, the only approach to repeat a step, or a series of steps, in an analysis was through repeated definition of the same step or steps multiple times. This resulted in reduced performance during all stages of a model (preprocessing, analysis, and output).

The use of step cycling provides an efficient mechanism to loop over the sequence of steps with significant performance improvements. It also provides mechanisms to end a step sequence prematurely based on certain user-specified criteria. A step cycle sequence can be preceded or followed by steps and/or another cycle sequence.



The figure above shows wear distance accumulation on the slider and the bottom block as the slider repeatedly moves back and forth. In this example, the sample wear cycle is modeled using two steps. In “Step-2” the slider moves to the right and returns to the center, while in “Step-3” the slider moves to the left and returns to the center. You can find additional details on this example in [Step Cycling for Contact Wear Simulations](#).

For more information, see *Step Cycling, Modeling Aging in Batteries, and Contact Wear*.

2024 FD03

Alternative Convergence Checks

A new set of alternative convergence controls are now available in Abaqus/Standard.

Benefits: In simulations that involve strong multiphysics coupling among a number of different fields, the alternative convergence controls, which are typically more relaxed compared to the default convergence controls, can result in better convergence characteristics and faster solution times with minimal effects on accuracy.

While the traditional convergence checks in Abaqus/Standard work well for structural simulations, they might be too restrictive for some multiphysics problems (such as battery electrochemistry simulations) that involve strong coupling among a number of different physical fields.

In earlier releases, the only way to modify the default convergence behavior in Abaqus/Standard was to specify nondefault tolerances on the maximum residuals and corrections during an increment. The new alternative convergence checks provide greater flexibility and can be used either globally (for all fields that are active in the analysis) or on an individual field basis. You can also modify the choice of the convergence checks at the step level.



Important: The use of this set of features requires some experience and testing to ensure that the alternative convergence measures produce results with an acceptable level of accuracy.

Table 1: Comparison of solver passes based on different convergence checks.

Model	Number of DOFs	Iteration Count		Gain
		Convergence Check = Strict	Convergence Check = Relaxed	
Model 1	792	1998	1662	1.20
Model 2	2232	673	357	1.88
Model 3	82692	156	128	1.22
Model 4	2380141	1417	958	1.48
Model 5	2764054	1998	1662	1.20
Model 6	4457894	545	395	1.38

For more information, see *Modifying the Default Convergence Checks*.

Adjoint Design Sensitivity Analysis

The adjoint design sensitivity functionality is enhanced significantly for eigenfrequency and direct steady-state dynamic analyses.

Benefits: You can perform the adjoint design sensitivity analysis faster, more capabilities are available, and the usability of the adjoint design sensitivity analysis is improved.

The following enhancements are available:

- Eigenfrequency adjoint design sensitivity analysis now supports the AMS eigensolver. In earlier releases, you could use only the Lanczos eigensolver for this analysis type. Now, you can run adjoint design sensitivity analyses for larger models and take advantage of parallel scaling in the AMS eigensolver.

- Direct steady-state dynamic adjoint design sensitivity analysis now supports topology and shape optimization. In earlier releases, you could use only sizing optimization for this analysis type.
- Direct steady-state dynamic adjoint design sensitivity analysis and eigenfrequency adjoint design sensitivity analysis now allow you to specify sizing and topology design variables for the same element.

For more information, see *Adjoint Design Sensitivity Analysis*.

State Space Solver for Transient Modal Dynamic Analysis

You can use the new state space solver to perform transient modal dynamic analysis for models with nondiagonal modal damping operators.

Benefits: The new solver is unconditionally stable and offers improved performance for models with many time increments and a moderate number of modes. It also supports coupled structural-acoustic analysis.

In earlier releases, the only solver available for transient modal dynamic analysis with nondiagonal modal damping was the conventional solver. The conventional solver uses a time integration scheme based on the assumption that the nondiagonal entries of the modal damping operator vary linearly within each time increment together with the loads. It calculates the approximate modal solution, and the solution accuracy depends on the time step size. In addition, the time integration algorithm is not unconditionally stable, particularly for models with a high level of damping. For common engineering simulations, the accuracy and stability of the conventional solver is usually acceptable. However, obtaining an accurate and stable solution using the conventional solver requires the use of very small time increments for specific cases. In addition, the conventional solver does not support coupled structural-acoustic analysis.

The new state space transient modal dynamic solver calculates the exact modal solution for systems with arbitrary modal damping for loads that vary piecewise linearly with time. This solver uses the complex eigenmodes of the unsymmetric modal state space operator to represent the real transient modal solution. The solution includes two phases: solving the unsymmetric eigenvalue problem for the state space operator at the “setup” phase and calculating the transient solution at the “apply” phase. The setup phase is the most computationally intensive part of the analysis. It can be costly for systems with many thousands of modes. However, the calculated solution with the state space solver is more accurate than for the conventional solver and the algorithm is unconditionally stable. In addition, the state space solver supports structural-acoustic transient modal analysis using the modal subspace built from the coupled structural-acoustic eigenmodes or from the uncoupled structural and acoustic eigenmodes.

For more information, see *Transient Modal Dynamic Analysis*.

Unit System Translation

The SIMULIA Co-Simulation Engine now supports unit system translation.

Benefits: You now have the flexibility to use the unit system best suited to your industrial workflow.

To avoid mistakes, it is best to perform coupled multiphysics and multiscale simulation where both models use the same units. Abaqus has no units built into it and assumes self-consistent units. Some third-party solvers might represent their internal data in a unit system different from the one in which the user defined the model.

The SIMULIA Co-Simulation Engine typically tabs into the solver and consequently exchanges field data in the solver’s unit system. This forces the Abaqus model to be in the unit system used by the other solver’s internal format and can lead to unconventional model specification. For example, consider a sequential analysis of a circuit board modeled in CST Studio Suite with results imported into Abaqus. You might employ the CGS system (centimeter-gram-seconds) to define the electromagnetic model. However, CST Studio Suite converts everything

to SI (International System) units to perform calculations. This forces the Abaqus model to be in SI units and to use meters as the length dimension for model definition (geometry, material data, load, etc.), which is an unconventional choice for this particular workflow.

Translation between unit systems is available only if the following are true:

- the unit system is defined in the Abaqus model and
- the other solver supports unit system translation through the co-simulation service.

For more information, see *Defining a Unit System for Translating Data with Other Solvers* and **UNIT SYSTEM*.

DMP Direct Sparse Solver Enhancements

The backward pass phase in the distributed memory parallel (DMP) direct sparse solver is improved.

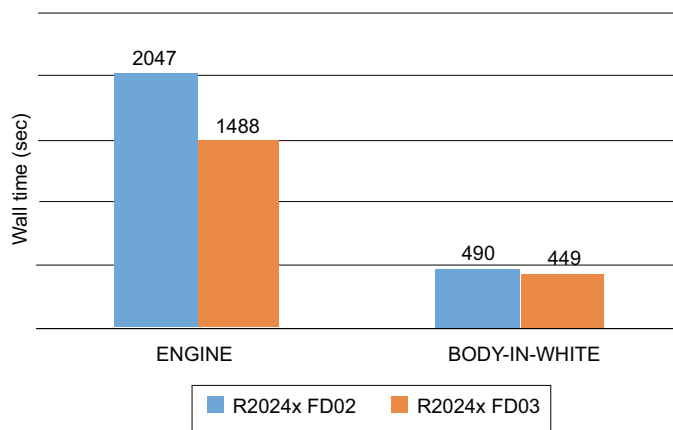
Benefits: The enhancements to the DMP direct sparse solver allow you to complete many analysis procedures faster.

Performance improvements are expected in workflows dominated by the backward pass; for example,

- static substructure generation using the conventional (not AMS-based) algorithm,
- linear perturbation static or steady-state dynamic analysis with multiple load cases, and
- eigenvalue extraction using the Lanczos eigensolver.

The improvements also eliminate cases where the solver could stop responding in certain DMP configurations. While this enhancement improves performance of the backward pass when the factorized matrix resides entirely in system memory, it might degrade performance for cases where the factorized matrix must be partially stored on disk.

To demonstrate performance improvements, two benchmark models are run on Intel Xeon Gold 6348 (2.6 GHz Ice Lake processor with 56 physical cores), with 2 NUMA nodes, and 2 sockets.



- **Model 1:** Static substructure generation for an engine model with 12M DOFs with 2028 retained DOFs run using a host configuration of $1 \times 1 \times 56$ (1 node, 1 mpi rank, 56 threads). This model shows 1.4x speedup in 2024 FD03 compared to 2024 FD02.

- Model 2: Direct steady-state dynamic analysis for a body-in-white car model with 3.6M DOFs with 20 frequency points and 120 load cases run using a host configuration of $2 \times 8 \times 7$ (2 nodes, 8 mpi ranks, 7 threads). This model shows 1.1x speedup in 2024 FD03 compared to 2024 FD02.

For more information, see *Direct Linear Equation Solver*.

MPI-Based Parallel Execution of the Lanczos Eigensolver

MPI-based parallel execution of the Lanczos eigensolver is now available.

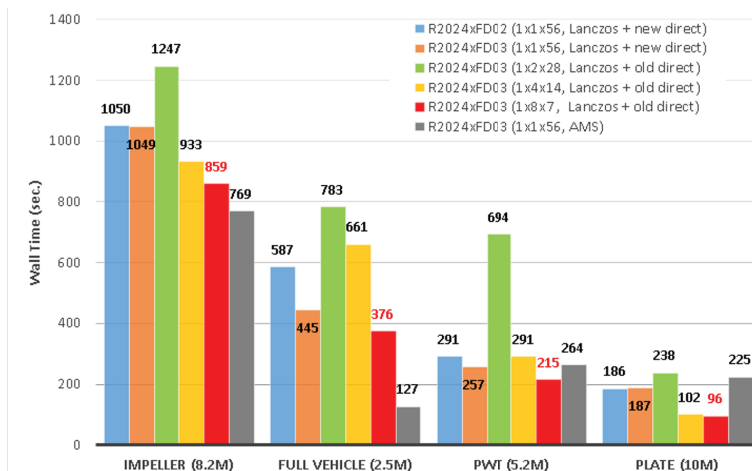
Benefits: Frequency extraction analysis using the Lanczos eigensolver in hybrid mode, which leverages both MPI and thread parallelism, runs significantly faster and can solve much larger problems by utilizing multiple compute nodes.

You can now run frequency extraction analyses using the Lanczos eigensolver in a hybrid mode that utilizes both MPI and thread parallelism. By leveraging both MPI and thread parallelism within a single compute node, as well as across multiple compute nodes on the HPC cluster, the new Lanczos eigensolver significantly enhances the performance of Lanczos eigensolver workflows.

The figure below shows the performance improvement of Abaqus/Standard using the new Lanczos eigensolver in 2024 FD03 for four benchmark models. Each column represents an Abaqus/Standard run time in a different host configuration. Because of the new enhanced thread-based direct solver released in 2024 GA, the Lanczos eigensolver uses the new enhanced thread-based direct sparse solver in shared memory parallel (SMP) mode and the DMP direct sparse solver available in earlier releases in hybrid mode.

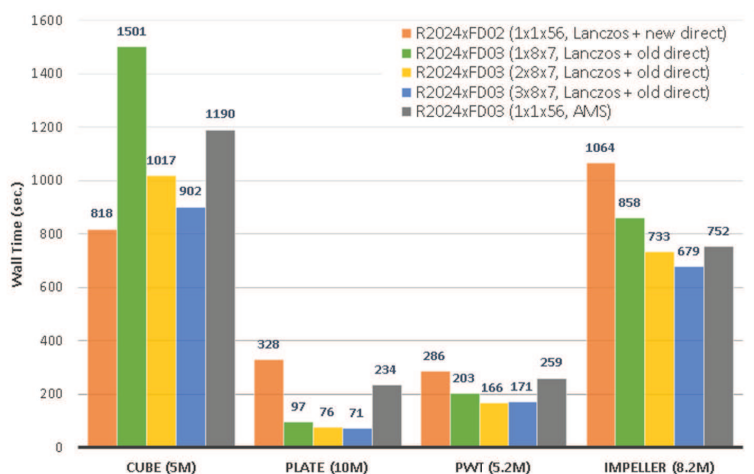
The figure illustrates the following:

- The Lanczos eigensolver in SMP mode in 2024 FD03 shows better performance than the previous version in 2024 FD02 for some of models due to the enhancements in the eigenmode transfer phase.
- The Lanczos eigensolver in hybrid mode in 2024 FD03 shows good parallel scaling using multiple MPI ranks on a single compute node. It also shows significantly improved performance compared to the eigensolver in 2024 FD02. For the full vehicle model with 2.5M DOFs, for example, Abaqus/Standard using a host configuration of $1 \times 8 \times 7$ in 2024 FD03 shows 1.6 times speedup compared to 2024 FD02.
- For all four benchmark models, the Lanczos eigensolver in hybrid mode ($1 \times 8 \times 7$) runs faster than the Lanczos eigensolver in SMP mode. It also runs faster than the AMS eigensolver with full recovery for some models in 2024 FD03. Therefore, the new Lanczos eigensolver in hybrid mode surpasses the performance limitations of the eigensolvers in SMP mode.



The figure below illustrates the performance improvement of the new Lanczos eigensolver when utilizing multiple compute nodes on the HPC cluster as follows:

- The new Lanczos eigensolver shows good parallel scaling using multiple compute nodes.
- The new Lanczos eigensolver using multiple compute nodes (that is, using more CPU cores and memory) runs significantly faster than the Lanczos eigensolver and the AMS eigensolver in SMP mode for all four models.
- For the impeller model with 8.2M DOFs, for example, Abaqus/Standard using a host configuration of 3 x 8 x 7 in 2024 FD03 runs 1.6 times faster than the previous version in 2024 FD02. It is even faster than the AMS eigensolver in 2024 FD03. This demonstrates that the new Lanczos eigensolver in hybrid mode using multiple compute nodes surpasses the performance limitations of the eigensolvers in SMP mode.



For more information, see *Parallel Execution in Abaqus/Standard*.

Parallel Execution for SPH Progressive Element Conversion

You can now run SPH progressive element conversion models in parallel.

Benefits: Parallel execution support can greatly improve the simulation performance of models that include SPH progressive element conversion.

Earlier releases of Abaqus/Explicit supported SPH progressive element conversion only in serial execution mode, limiting the usability of this feature for broad applications. This release supports parallel execution for SPH progressive element conversion, which enables you to take full advantage of the intrinsic strengths of both the Lagrangian finite element and SPH methods.

Figure 1 shows a parallel run example for a model of a bird striking an airplane engine blade. The bird structure consists of 480 C3D8R elements, with an EOS material definition that can be converted to around 100k SPH particles based on a strain conversion criterion. Once the strain for a certain C3D8R element meets the user-defined conversion criterion, the element is converted to SPH particles to sustain more severe deformation. The graphs in *Figure 2* show the parallel execution speedup as function of the number of processors, illustrating reasonably good scaling.

The parallel speedup might vary for different models depending on the uniformity of the SPH particle distributions. It is recommended to use more uniform-sized parent elements (preferably C3D8R) for the SPH progressive element conversion to achieve better accuracy and parallel scalability.

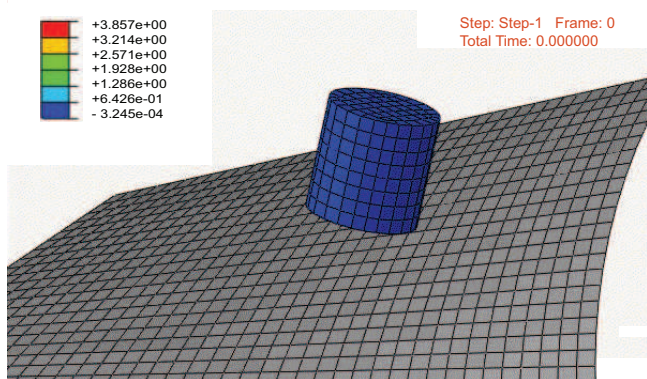


Figure 1: Bird strike on an airplane engine blade using strain-based SPH element conversion.

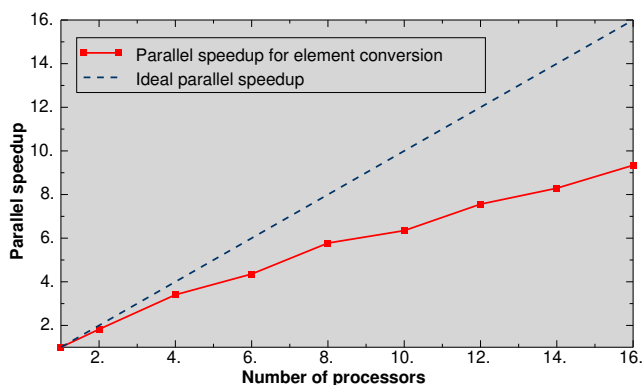


Figure 2: Parallel speedup for the bird strike model.

Fatigue Crack Growth Analysis Using Field Import-Based Submodeling

You can now import external fields in a linear elastic fatigue crack growth analysis.

Benefits: You can now predict fatigue crack growth in submodels through external field import by mapping fields from the global model to the boundary of the submodel. This allows you to use global models with multiple steps.

In the traditional submodeling approach, the submodel can be driven only by results from a single step in a global model.

For more information, see *General Capability for Importing External Fields*, *Linear Elastic Fatigue Crack Growth Analysis*, and *Node-Based Submodeling*.

2024 FD02

Element Design Responses for Transient Adjoint Design Sensitivity Analysis

Transient sensitivity analysis now supports element design responses based on stress and plastic strain as well as user-defined responses.

Benefits: Element design responses, such as stresses and various measures of plastic strain, allow design optimization for transient dynamic workflows to set constraints on the maximum von Mises stress and/or various measures of the plastic strain.

In earlier releases, Abaqus limited the list of available design responses in a transient sensitivity analysis primarily to nodal responses. The rate of internal work was the only available element-level response. The list of element design responses is now expanded significantly to include different stress and/or plastic strain measures.

In addition, you can use user subroutine *UELEMDRESP* to define a user-defined element design response, such as a failure criterion, in a transient optimization.

For more information, see *Adjoint Design Sensitivity Analysis* and *UELEMDRESP*.

Specifying Design Responses in an Adjoint Design Sensitivity Analysis

You can now specify design responses in an adjoint design sensitivity analysis. In addition, you can prevent sensitivity calculations and output that might not be important for design optimization.

Benefits: You can manage the specification of design responses more effectively and prevent the output database from becoming too large.

The list of available operators for node and element design responses now includes the maximum absolute and the minimum absolute operators. These operators can be helpful in situations such as the following:

- In the context of node responses, the maximum absolute operator is useful when the nodal displacements for a node set that contains many nodes should be constrained to be less than a certain value. Without the maximum absolute operator, you must define one node response for each node in the node set, leading to many node responses (and corresponding adjoint sensitivities) in the output database. When you use the maximum absolute operator, you need to define only a single design response that uses this design objective or constraint.
- The minimum absolute operator is useful when the reaction force for a node set that contains many nodes must be larger than a certain value. Without the minimum absolute operator, you must define one node response for each node in the node set, leading to many node responses (and corresponding adjoint sensitivities)

in the output database. When you use the minimum absolute operator, you need to define only a single design response that uses this design objective or constraint.

- You can now specify lower and upper bounds on the values of a design response outside of which Abaqus will not compute the adjoint sensitivities. By default, Abaqus writes the design response and the corresponding adjoint sensitivities to the output database for all design responses that do not have any prescribed bounds for the design response definition. In some situations, the sensitivities are necessary only if the value of a design response is within a certain range. In these cases, specifying the lower and upper bounds on the design response values provides a convenient way to limit the sensitivity calculations and the size of the output database.

For more information, see *Adjoint Design Sensitivity Analysis*.

Structure-to-Structure Co-Simulation Using Abaqus/Standard and Abaqus/Explicit

The keyword interface has changed for supporting structure-to-structure co-simulation using Abaqus/Standard and Abaqus/Explicit.

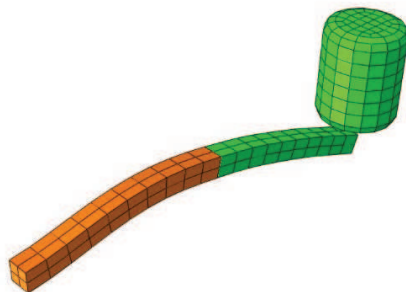
Benefits: The new keyword interface is aligned with other co-simulation products and supports improved interface computations that improve structure-to-structure co-simulations.

Co-simulation between Abaqus/Standard and Abaqus/Explicit exhibits strong physics coupling and requires special treatment at the co-simulation interface. The enhanced subcycling method (see *Enhanced Subcycling Method for Interface Calculations*), which improves solution robustness and is more computationally efficient, was introduced in an earlier release. The enhanced subcycling method is recommended over the original subcycling method (see *Original Subcycling Method for Interface Calculations*) and the lockstep method (see *Lockstep Method for Interface Calculations*).

The original subcycling method and the lockstep method used the **CO-SIMULATION, PROGRAM=ABAQUS* option. Since these are no longer the recommended interface treatments, the *PROGRAM=ABAQUS* parameter is being deprecated. In addition, the **CO-SIMULATION CONTROLS* option, which selects between the original subcycling and lockstep algorithms, is also being deprecated.

You can still select any of the above interface treatment methods by specifying the fields exchanged explicitly and by using the appropriate co-simulation service configuration file, as described in *Structural-to-Structural Co-Simulation*.

The image below shows a cantilever beam impacted by an object. The beam is split into two domains: the orange portion is modeled in Abaqus/Standard, and the green portion is modeled in Abaqus/Explicit.



For more information, see *Structural-to-Structural Co-Simulation*.

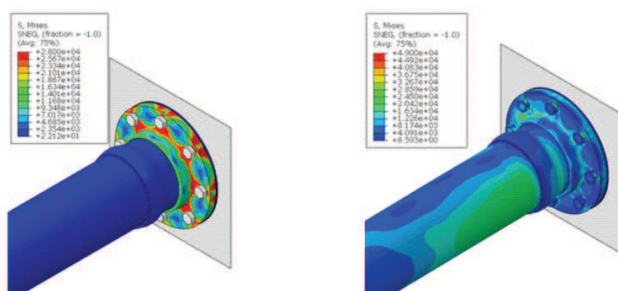
Full-Model Input for Abaqus Import

You can use the full-model input format to specify both new and imported model data in an Abaqus/Standard or Abaqus/Explicit import analysis.

Benefits: The full-model input format allows flexibility in import feature management; for example, partial import of element sets of a section, import of previously rigid element sets as deformable, and import analysis using the SIM results format.

When you use the full-model input format, you give imported model data (such as nodes and elements, node sets and element sets, and sections and materials) in the input. This provides complete model information used in the import analysis.

By using the full-model input format, you can perform certain import workflows (such as importing a previously rigid element set as deformable) simply by omitting the rigid body definition of the element set in the import input. The following example illustrates this approach. The bolts are defined as rigid in the first analysis (left), and they are imported as deformable in the import analysis (right).



For more information, see *About Transferring Results between Abaqus Analyses*.

Undrained Pore Fluid Flow and Stress Analysis

You can now perform an undrained pore fluid flow and stress analysis in Abaqus/Explicit.

Benefits: You can account for the presence of a permeating pore fluid and its effect on the mechanical response of a saturated granular solid skeleton under dynamic loading conditions.

A porous medium typically consists of a solid skeleton of numerous solid grains that are “loosely packed” and a permeating pore fluid that occupies a system of interconnected pores among these solid grains. You can ignore the effect of fluid flow in situations when the loading rate is significantly faster than the rate at which pore fluid can drain from the porous medium. This corresponds to the undrained conditions.

The undrained pore fluid flow and stress analysis functionality in Abaqus/Explicit is intended for the simulation of fully saturated porous media under undrained conditions. The functionality enables modeling the increase in stiffness of the soil in applications such as trenching, gouging, and pile driving. In these applications, you can ignore fluid diffusion terms when the rate of loading is fast compared to the characteristic time of the diffusion process.

For more information, see *Undrained pore fluid flow and stress analysis*.

2024 FD01

Iterative Solver Performance Improvements

A new domain decomposition algorithm improves iterative solver convergence rates and shortens the overall solution time.

Benefits: The iterative solver is now able to converge on models on which it could not converge in earlier releases (for example, automotive battery structural simulations) or on models for which convergence was prohibitively slow (for example, electric engine analyses).

The new domain decomposition algorithm takes into account the connectivity of the finite element model, the element formulation details, and the material properties. It creates a better clustering of elements with similar properties and separates out clusters with dissimilar properties. This makes it easier for the iterative solver to find a smooth solution in individual aggregates, which results in improved convergence rates overall. The algorithm optimizes the load balance between domains for large models in parallel runs by accounting for the actual computational costs of different elements. The solution time reduction due to the improved domain decomposition varies from model to model. The improvement can be moderate for some finite element models, but it can be significant for large-scale models with complicated geometry using higher-order element formulations.

The figure below illustrates the improved performance of the iterative solver for four selected finite element models:

- Model 1: airplane fuselage structure; shell-dominated finite element model with 48.3 million degrees of freedom.
- Model 2: powertrain model with 23.6 million degrees of freedom meshed with linear continuum elements.
- Model 3: powertrain model with 11.2 million degrees of freedom meshed with quadratic continuum elements.
- Model 4: battery model with 46.0 million degrees of freedom, meshed with linear continuum, shell, and connector elements.

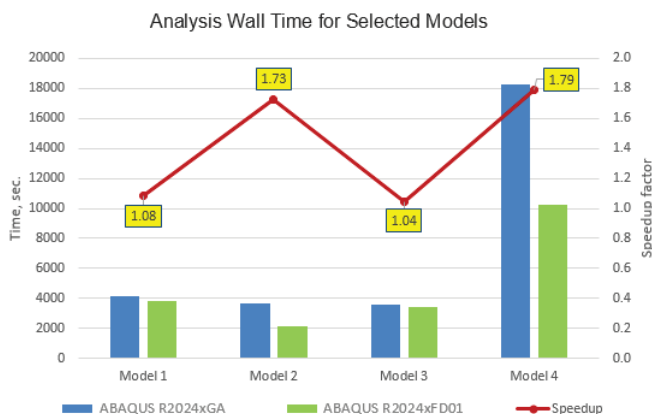


Figure 3: Performance improvements from using the new domain decomposition with the iterative solver.

For more information, see *Iterative Linear Equation Solver*.

Generating Reduced Bases for Use with Nonlinear Flexible Multi-Body Systems

A new analysis technique in Abaqus/Standard reads operator data from upstream analyses and generates reduced bases for use in downstream nonlinear flexible multi-body system (MBS) simulations with Simpack.

Benefits: This streamlines the process for preparing MBS simulations that capture large deformation effects with high accuracy and fast run times.

A reduced basis is a set of basis vectors that can represent solutions from preceding analysis cases when they are superimposed with according scaling factors. You can now generate reduced bases for use in downstream nonlinear flexible MBS simulations with Simpack as follows:

1. Define a nonlinear static analysis case that ramps up a large deformation of a flexible component in a shape that is characteristic for its behavior in downstream system simulations. For example, apply a distributed load that acts on the surface of a wind turbine rotor blade in the flap-wise direction, or define a spring support for a twist beam axle and apply a twisting load at the wheel attachment points.
2. Output the operator data at various deformation levels.
3. Add natural frequency extraction steps at various deformation levels.
4. Add the reduced basis generation step.

The reduced basis generation step computes the reduced basis from the solution data and computes the auxiliary operator data from all the preceding analysis steps. The reduced basis and projected operator data are written to a SIM (.sim) file. Optionally, you can write a Simpack Flexible Body Interface (.fbi) file with a complete Simpack nonlinear flexible body definition that you can assign directly to a nonlinear flexible body in Simpack.

For more information, see *Reduced Basis Generation* and **REDUCED BASIS GENERATE*.

Random Response Analysis Performance Improvements

Random response analyses now have improved performance for the element results computation and output.

Benefits: The improvements reduce the Abaqus/Standard analysis wall time for random response analyses, particularly for large models with extensive element field output requests.

The enhancement substantially improves performance of the following phases of the random response analysis:

- Reading the eigenfrequency extraction stresses.
- Computing the element results. New scalable algorithms perform the element output computations.

In addition, you can now use the frequency split technique to run a random response analysis on models that do not require calculations of the root-mean-square variables.

[Figure 4](#) illustrates the wall time reductions for three industrial models, and [Figure 5](#) illustrates the reduction in the wall time for the element results computation phase for the same three models.

The models and results are as follows:

- Model 1 is comprised of 2.3 million elements containing approximately 9 million degrees of freedom. The random response analysis was performed using 83 eigenmodes and 840 frequency points. Stress field output was requested in all increments.

The Abaqus/Standard wall time was reduced approximately 5.25 times, while the element results computation phase was sped up approximately 9 times.

- Model 2 is comprised of 10.8 million elements containing approximately 53 million degrees of freedom. The random response analysis was performed using 38 eigenmodes and 20 frequency points. Stress and strain field output was requested in all increments.

The Abaqus/Standard wall time was reduced approximately 3 times, while the element results computation phase was sped up approximately 10 times.

- Model 3 is comprised of 5 million elements containing approximately 25 million degrees of freedom. The random response analysis was performed using 110 eigenmodes and 38 frequency points. Stress and Mises stress field output was requested in 0.6 million elements at all frequency points.

The Abaqus/Standard wall time was reduced approximately 1.4 times, while the element results computation phase was sped up approximately 3.6 times.

Figure 6 shows the result of applying the frequency split technique to Model 1. The 840 frequency points used for the random response analysis were split into 16 batches. The total Abaqus/Standard wall time (that includes the wall time of a frequency extraction step and the largest wall time of the random response analyses) was reduced approximately 3.75 times, while the element results computation phase was reduced approximately 11 times.

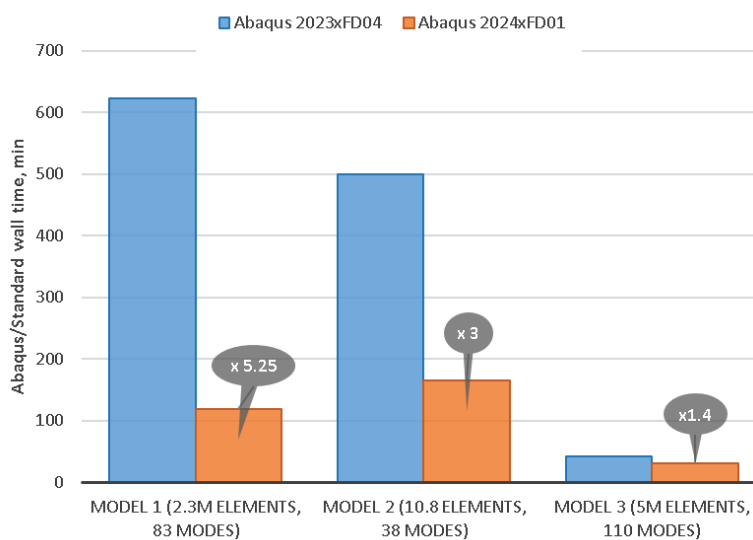


Figure 4: Wall time improvements for running a random response analysis in Abaqus/Standard for three industrial models.

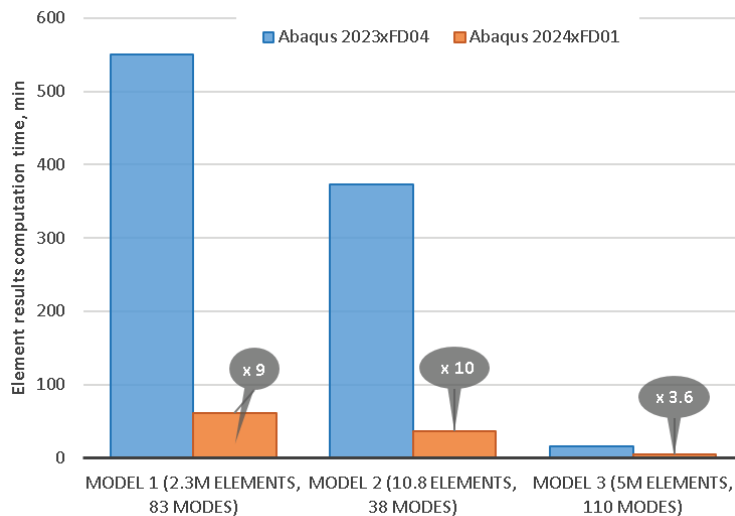


Figure 5: Performance improvements for element results computation time for running a random response analysis in Abaqus/Standard for three industrial models.

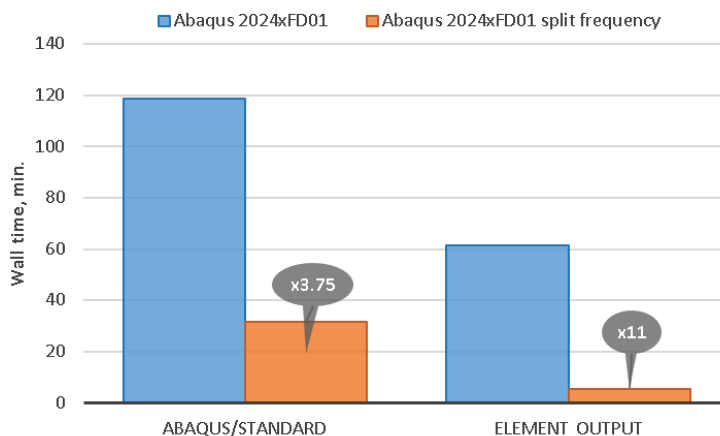


Figure 6: Performance improvements for running the random response analysis using the split frequency technique for Model 1.

For more information, see *Random Response Analysis*.

AMS Eigensolver Performance Improvement

The performance of the frequency extraction analysis using the AMS eigensolver has been improved for models that require full eigenmodes and substructure modes.

Benefits: The AMS full recovery performance improvement significantly reduces the analysis run times for an AMS frequency extraction analysis and for an AMS-based substructure generation analysis requesting full eigenmodes.

The performance of the following phases related to full eigenmodes recovery is significantly improved for the AMS eigensolver workflow including the AMS-based substructure generation procedure:

- AMS recovery phase
- Constraint transformation and recovery phases
- Substructure recovery matrix output phase

The figure below illustrates the performance improvement of Abaqus/Standard in Abaqus2024 FD01 for four industrial models. In this figure, the blue columns represent the Abaqus2023 GA run time, and the orange columns represent the Abaqus2024 FD01 run time.

The models and results are as follows:

- Model 1 is a heavy-duty engine model with 15.2 million degrees of freedom, extracting 10 eigenmodes below 100Hz and 5,704 substructure modes for 5,704 retained degrees of freedom.

Abaqus2024 FD01 shows a 36% performance improvement over Abaqus2023 GA.

- Model 2 is a powertrain model with 26.3 million degrees of freedom, extracting 6 eigenmodes below 500Hz and 1,512 substructure modes.

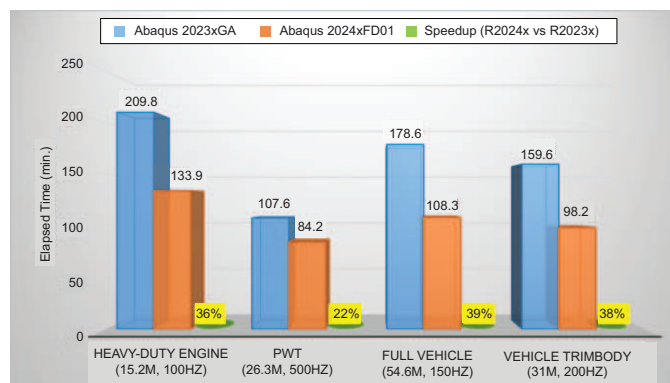
Abaqus2024 FD01 shows a 22% performance improvement over Abaqus2023 GA.

- Model 3 is a full vehicle model with 54.6 million degrees of freedom, extracting 1,630 eigenmodes below 150Hz and 1,984 substructure modes.

Abaqus2024 FD01 shows a 39% performance improvement over Abaqus2023 GA.

- Model 4 is a vehicle trim body model with 31 million degrees of freedom, extracting 2,598 eigenmodes below 200Hz.

Abaqus2024 FD01 shows a 38% performance improvement over Abaqus2023 GA.



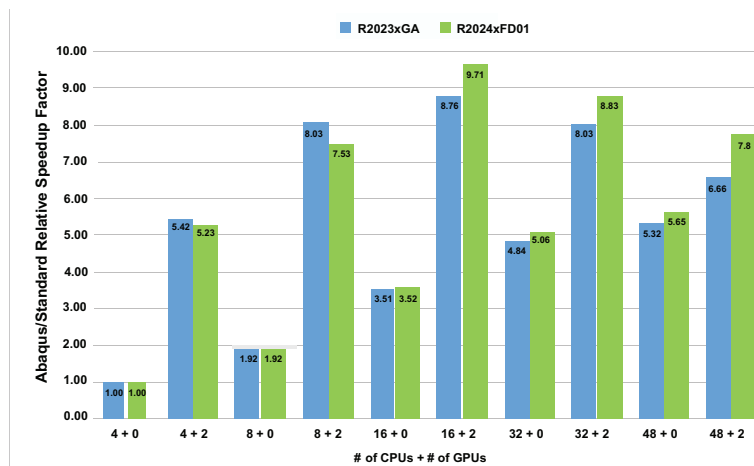
For more information, see *Natural Frequency Extraction*.

AMS Eigensolver GPU Acceleration

Robustness and performance of the AMS eigensolver GPU acceleration is improved for frequency extraction analyses with new NVIDIA GPU cards. New GPU acceleration functionality speeds up modal damping projection in the AMS recovery phase.

Benefits: Analyses using the AMS eigensolver run faster using multiple GPUs.

The figure below shows the GPU acceleration performance of Abaqus/Standard for a 7.6 million degree of freedom stator model that requires 2.3×10^{15} floating-point operations to factorize four system operators (including viscous and structural damping matrices). The blue columns represent the Abaqus2023 GA run time, and the green columns represent the Abaqus2024 FD01 run time. The performance data was obtained on a machine equipped with Intel Xeon Gold 6336Y (Ice Lake, 2.4GHz, 48 cores) processors, 512 GB physical memory, and 2x NVIDIA A100 (40 GB memory) cards.



Abaqus/Standard using the AMS eigensolver shows good parallel scaling up to 48 CPUs. However, using 16 CPUs and 2 GPUs results in the best performance of the GPU acceleration for this model.

For more information, see *Parallel Execution in Abaqus/Standard*.

Scalable Spectral Solver for Mode-Based Steady-State Dynamics Analysis

A new scalable spectral solver is available for common cases of mode-based steady-state dynamics analysis.

Benefits: Using the new solver can significantly reduce the analysis time for damped finite element models with general structural damping and specific viscous damping properties.

You can use the new spectral solver for the special cases of mode-based steady-state dynamic simulations. For all cases, you can apply any kind of structural damping in the model. The special cases are defined by the complementary viscous damping properties.

Case 1	No viscous damping	No material, global, or modal viscous damping
		No dashpot elements, connector damping, or substructures with viscous damping
		No matrix input data for the viscous damping
		No other possible sources of the viscous damping
Case 2	Low rank viscous damping operator	Viscous damping is defined locally in the model, for example, using dashpot elements,

		connector damping, or substructures with viscous damping
		Structural-acoustic coupled steady-state dynamic analysis using uncoupled eigenmodes when the number of acoustic modes is significantly smaller than the number of the structural modes
		Modal viscous damping defined for selected modes

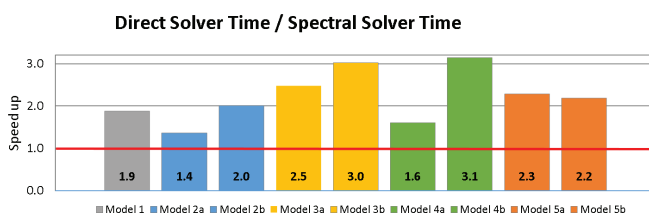
You can use the spectral solver for mechanical, acoustic, and structural-acoustic analyses. Using the spectral solver can be beneficial for models with the mentioned special damping properties when many eigenmodes and many points in the frequency domain are used for the frequency response calculation.

The spectral solver algorithm consists of two analysis phases: the Setup Solver Phase and the Apply Solver Phase. The computational cost of the Setup Solver Phase depends on the number of the eigenmodes, and it can be significant when that number is high. The computational cost of the Apply Solver Phase is proportional to the number of the frequency points specified for the frequency response calculation, and the solution calculation at a frequency point is relatively cheap. Therefore, the spectral solver can be less effective for the analyses with a small number of frequency points (when the setup phase dominates), but it can be very effective for the same model when more frequency points are used.

The figure below shows the performance benefits from using the new spectral modal solver over the conventional (default) solver that solves the system of equations with a dense complex matrix and multiple right hand sides at every frequency point. It compares the wall-time ratios for a steady-state dynamic analysis using the conventional and the new spectral solvers for the following models:

Model		DOF	Load Cases	Eigenmodes		Frequency Points	Damping Properties	Cores
1	Body-in-white	3.6 million	120	12200		1000	Material structural damping; no viscous damping	40
2	Full vehicle	9.7 million	165	8800 uncoupled eigenmodes including 1000 acoustic modes		1000	Material structural damping; viscous damping —6 dashpots	40
						2000		
3	Body-in-white	35.6 million	21	33800		1000	Material structural damping; viscous damping —70 dashpots	40 56
4	Body-in-trim	2.6 million	110	11000+	Uncoupled structural and	1000	Material structural	

Model		DOF	Load Cases	Eigenmodes		Frequency Points	Damping Properties	Cores
					acoustic eigenmodes		damping; viscous damping — 6 dashpots	
					Fully coupled structural-acoustic eigenmodes			
5	Fuselage model	48.3 million	120	21300		1000	Material structural damping; no viscous damping	40
								56



Beam-Based Submodeling from Global Beam Elements to Shell Elements

You can now perform beam-to- shell submodeling.

Benefits: Using shell elements in a submodel allows you to gain more detailed insight on results of interest for a region of a thin-walled beam that is modeled with beam elements in a global model.

Beam-to-shell submodeling is often a good strategy to study details in one or more local regions if a global model uses beam elements. [Figure 7](#) shows a local shell-element model superimposed with beam elements from a global model. The shell thickness is rendered in [Figure 7](#). This simple example is provided to demonstrate the concept; typically, the global model would be larger and more complex than shown here.

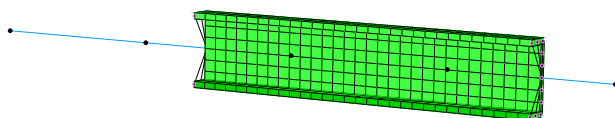


Figure 7: Superimposed global (beam-element) and local (shell-element) models for a beam with a channel cross section.

Beam-to-shell submodeling is similar to the beam-to-solid submodeling capability introduced in Abaqus 2023 FD03; in both cases distributing couplings are used to impose conditions from the global model at submodel cuts. For comparison, [Figure 8](#) shows a local solid-element model superimposed with beam elements from a global model. Representations of distributing couplings at submodel cuts are shown in [Figure 7](#) and [Figure 8](#).

Reference nodes of these distributing couplings are used to impose conditions from the global model on the submodel. [Figure 9](#) shows the global model with beam cross-sections rendered.

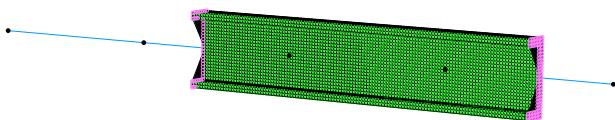


Figure 8: Superimposed global (beam-element) and local (solid-element) models for a beam with a channel cross section.

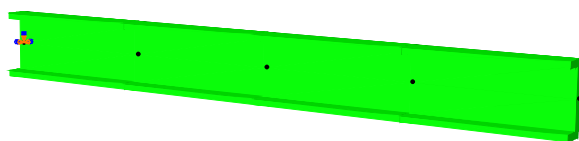


Figure 9: Rendering of the channel cross section for beam elements of the global model.

[Figure 10](#) provides a comparison of displacement contour plots for the beam-element global model and each of the submodels.

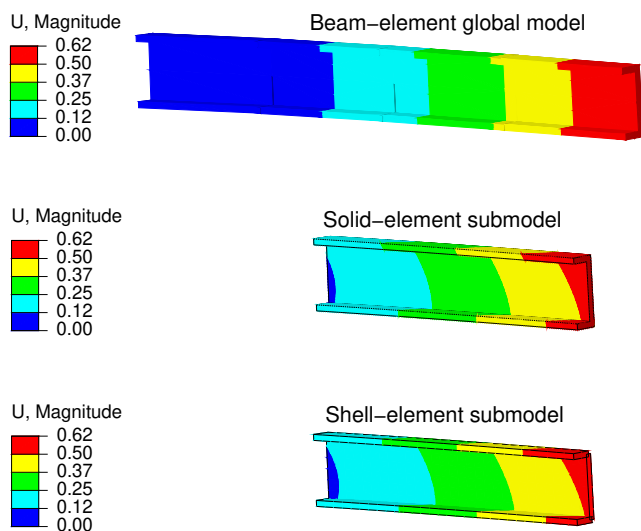


Figure 10: Contour plots of displacement magnitude for the beam-element global model and each of the submodels.

[Figure 11](#) shows Mises stress contour plots for each of the submodels.

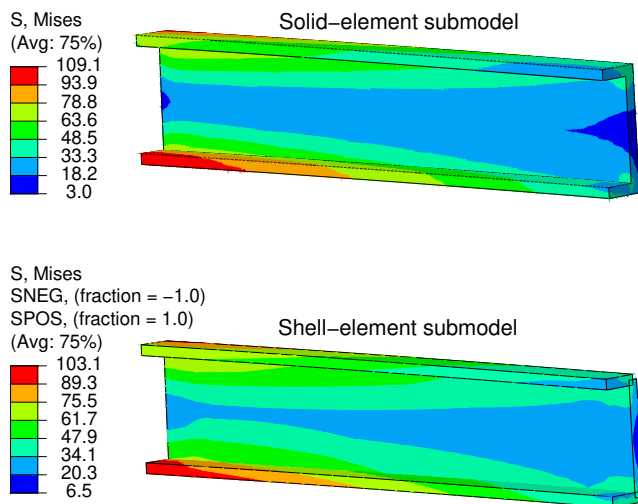


Figure 11: Contour plots of Mises Stresses for each of the submodels.

For more information, see *Beam-Based Submodeling*.

2024 GA

Python 3

Abaqus Python is now based on Python 3.10 instead of Python 2.7. For more information about this major change and what it means for Abaqus users, visit go.3ds.com/AbaqusPython.

Benefits: This update brings Abaqus Python into conformance with a version of Python that is supported in the open-source community.

Formation of Tie Constraints in Import Analysis

In an Abaqus/Explicit import analysis, Abaqus/Explicit now forms tie constraints using the original instead of the updated configuration if the definitions of both the main and secondary surfaces to be tied are unchanged from the original analysis.

Benefits: The new protocol makes it easier to establish initial equilibrium due to tie constraints in the imported model.

The new protocol prevents the formation of new tie constraints for nodes on the surfaces that were previously untied. Such new tie constraints, if formed, can adversely affect the consistency of the imported model. For example, if the previously untied nodes are in a state of contact, tying these nodes results in deletion of the contact state associated with the nodes.

The new protocol also guarantees formation of the same tie constraints for the surfaces in the import analysis as in the original analysis. It is used regardless of whether you specify the original or the updated configuration as the reference configuration for import.

For more information, see *About Transferring Results between Abaqus Analyses*.

Adjoint Sensitivity Analysis for Sizing Optimization in Direct Steady-State Dynamic Analysis

You can now compute adjoint sensitivities for select node responses defined over node sets for a direct steady-state dynamic analysis to perform sizing optimization.

Benefits: You can now optimize for the steady-state response (direct-solution) of a structure subjected to harmonic excitations.

Abaqus/Standard now supports sizing optimization for select design variables (beam radii and shell thicknesses) in direct steady-state dynamic analyses for select design responses (nodal displacements/velocities/accelerations/rotations). You can define the range of frequencies over which you wish to compute the sensitivities for a particular design response.

You can use this new capability only for jobs run on a single domain and for node sets with nodes corresponding only to independent degrees of freedom.

For more information, see *Adjoint Design Sensitivity Analysis*.

Benchmarks

This page describes recent changes in Abaqus Benchmarks.

2024 FD03

Fatigue Crack Growth Analysis Using Field Import–Based Submodeling Approach

New tests verify fatigue crack growth analysis using both the traditional node-based submodeling approach and the field import–based submodeling approach.

Benefits: The tests verify the capability in Abaqus/Standard.

A single edge notch subjected to pure mode II cyclic fatigue loading is carried out using both the traditional node-based submodeling approach and the field import–based submodeling approach. Similar results are obtained when comparing the crack profiles obtained using both approaches.

For more information, see *Crack propagation of a single-edge notch simulated using XFEM*.

Using Low-Density Foam with Eulerian Elements and ALE Analysis

New tests verify the use of low-density foam with Eulerian elements and ALE adaptive meshing.

Benefits: The tests verify the capability in Abaqus/Explicit.

For more information, see *Indentation with different materials*, *Wave propagation with different materials*, and *Adaptivity patch test with different materials*.

Constraints

This page describes recent changes in Abaqus Constraints.

2024 FD01

Conversion of Select Distributed Couplings to Tie Constraints

Abaqus/Explicit now converts distributing couplings to a tie constraint by default when the distributing couplings have cloud nodes associated with a single element facet and when the reference node of the coupling is not a node of a connector element.

Benefits: This enhancement avoids the creation of overlapping distributed couplings (which introduce a small amount of artificial mass for numerical purposes) when modeling fasteners using solid elements.

Some workflows involve fastening two surfaces together with the stiffness of the connection represented by a hexahedral element instead of a connector element, as shown in [Figure 1](#). Each node of the hexahedral element acts as a reference node of a distributing coupling whose cloud nodes are associated with a single shell element, as shown in [Figure 2](#). This approach results in some shell nodes participating in two or four distributing couplings. Abaqus/Explicit generally introduces artificial mass to nodes acting as clouds nodes in multiple distributing couplings. By changing the default behavior for automatic conversion of distributing couplings to tie constraints, Abaqus/Explicit reduces the likelihood of adding artificial mass to a model.

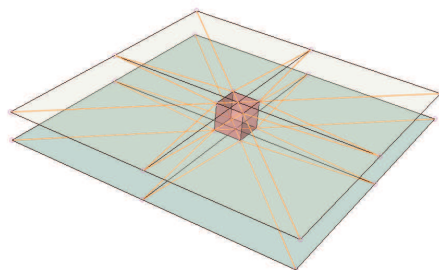


Figure 12: A fastener construct involving a hexahedral element representing the faster stiffness.

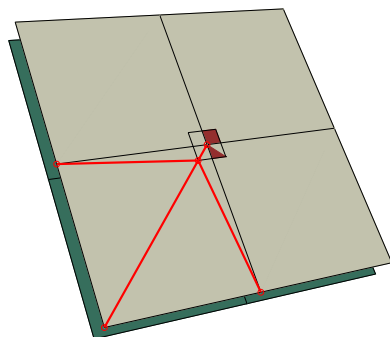


Figure 13: Highlighted connectivity for one of eight user-defined distributing couplings associated with a fastener connection.

Constraint coefficients are somewhat different for distributed couplings than they are for tie constraints in such cases. [Figure 3](#) provides perspective on these differences. A downward unit force is applied at node 101. Node 101 is connected to a single square face with a distributed coupling for the case on the left versus a tie constraint for the case on the right. The four nodes of the square face are fixed with boundary conditions. Node 101 corresponds to the position one-tenth along the diagonal line from nodes 1 to 4. The vertical reaction forces (RF3) for nodes 1 through 4 are 0.65, 0.25, 0.25, and -0.15 for the distributing coupling case; and for the tie constraint case these reaction forces are 0.81, 0.09, 0.09, and 0.01, respectively. These sets of reaction forces are statically equivalent (both sets of reaction forces sum to 1.0 and do not generate a moment about the location of node 101). Because the constraint coefficients are different in the two approaches, some differences in the local behavior around the constraint are expected due to this conversion.

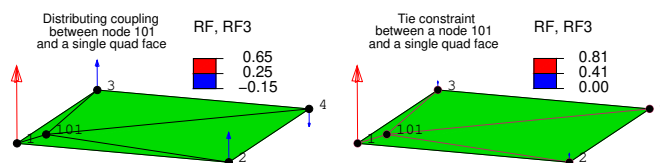


Figure 14: Comparison of reaction forces for a distributed coupling versus a tie constraint acting between node 101 and a single quad face, with unit downward load applied at node 101 and boundary conditions acting at nodes 1 through 4.

You can override this conversion from distributed couplings to tie constraints if desired.

The recommended technique for modeling compliant fasteners is to use connector elements instead of hexahedral “nuggets.” This enables the use of larger numbers of cloud nodes per coupling, which reduces mesh sensitivity without having nodes act as cloud nodes in multiple overlapping distributed couplings.

For more information, see *Converting a Distributing Coupling Attached to a Single Element Facet to a Tie Constraint*.

Elements

This page describes recent changes in Abaqus Elements.

2024 FD03

Bending Stabilization for Full-Integration Linear Elements

You can now control the activation and the level of bending stabilization for full-integration linear elements.

Benefits: Bending stabilization improves robustness for models using full-integration linear elements modeled with hyperelastic, hyperfoam, low-density foam, or user-defined materials and subjected to large bending deformations.

For full-integration linear elements modeled with hyperelastic, hyperfoam, low-density foam, or user-defined materials, spurious unstable patterns might develop in some situations due to excessively soft bending behavior. The formulation for this class of problems includes a bending stabilization term to improve the element performance. In earlier releases, the bending stabilization was active by default for built-in hyperelastic, hyperfoam, and low-density foam materials, but it was not active for user-defined materials. This could lead to different outcomes (convergence behavior and results) when you modeled the same material behavior using either a built-in material model or a user-defined material model. In addition, you had no control over the activation/deactivation of the stabilization algorithm or on the degree (magnitude) of stabilization.

The example below shows a two-dimensional rubber ring compressed by a rigid plate against a base that is also made out of rubber. [Figure 1](#) shows the results from a simulation that does not make use of the bending stabilization term. The analysis runs into convergence difficulties and fails to converge following the initiation of self-contact in the ring. Conversely, [Figure 2](#) shows the results from a simulation that utilizes bending stabilization. In this situation, the analysis completes. [Figure 3](#) shows the force-displacement behavior of the rigid plate's reference node for the two models. The two responses are in close agreement until convergence is lost in the model without stabilization, demonstrating that apart from improving convergence, the bending stabilization's impact on overall model response is small. The bending stabilization term is very important for problems involving a relatively soft material subjected to large bending deformations.

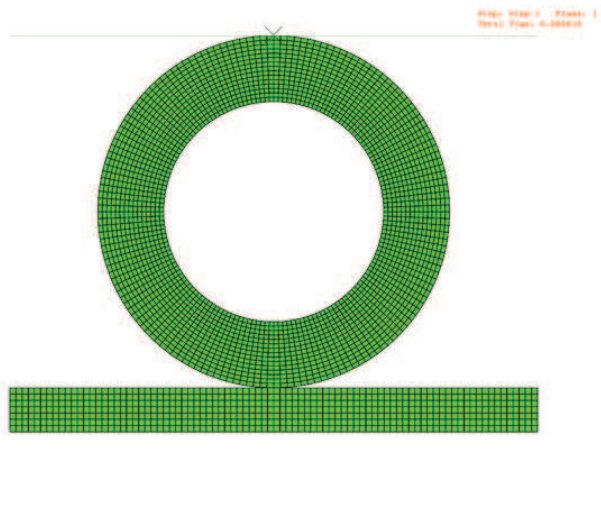


Figure 15: Modeling of a compressed O-ring with inactive bending stabilization.

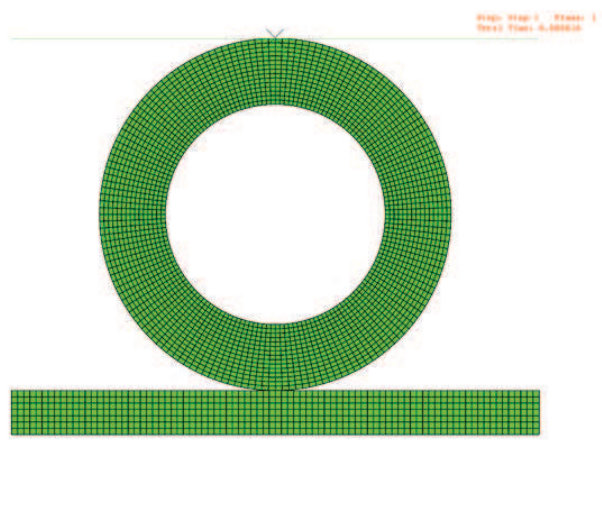


Figure 16: Modeling of a compressed O-ring with active bending stabilization.

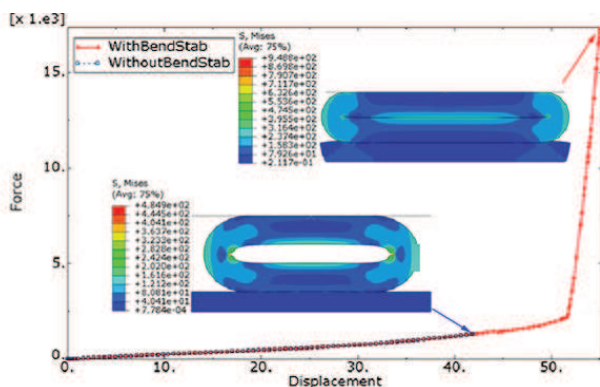


Figure 17: Force-displacement response of plate reference node with and without bending stabilization.

For more information, see *Using Section Controls for Bending Stabilization*.

2024 FD01

Varying Contact Angles for SLIPRING Connectors

You can now request that Abaqus/Standard compute the contact angle of a SLIPRING connector at the beginning of the analysis and update it through the analysis.

Benefits: If the slipring connector models a belt-pulley assembly, basing the calculations on the current value of the contact angle more accurately calculates the frictional force between the pulley and the belt.

In earlier releases, Abaqus/Standard assumed the SLIPRING contact angle to be either a constant (specified) value or zero (the default).

Figure 4 shows a belt-pulley system between nodes 1, 2, and 3. The belt is modeled using two slipring connector elements and passes over a pulley at node 2. The frictional force between the belt and the pulley depends on the contact angle between them. In this example, the contact angle changes as nodes 1 and 3 are displaced from their original positions. Abaqus/Standard now accounts for the change in this contact angle for the friction calculations.

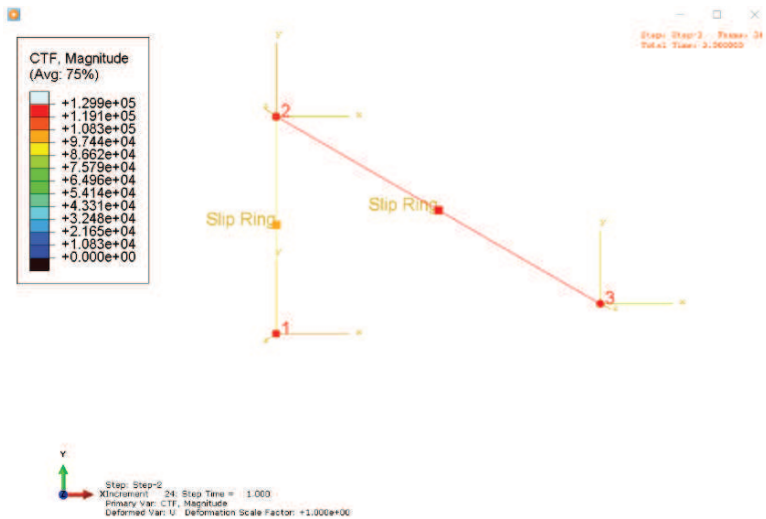


Figure 18: SLIPRING connection type with varying contact angle.

For more information, see *SLIPRING*.

Example Problems

This page describes recent changes in Abaqus Example Problems.

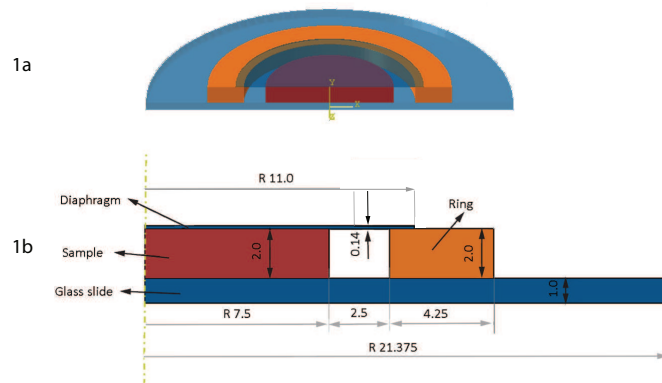
2024 FD04

Modeling the Watts Test of Adhesives

A new example demonstrates how to model the curing process of adhesives.

Benefits: The tests demonstrate the adhesive curing process modeling capability in Abaqus/Standard.

The Watts test measures the polymerization shrinkage of materials. In the Watts test, a disc-shaped specimen is sandwiched between two glass plates, as shown below.



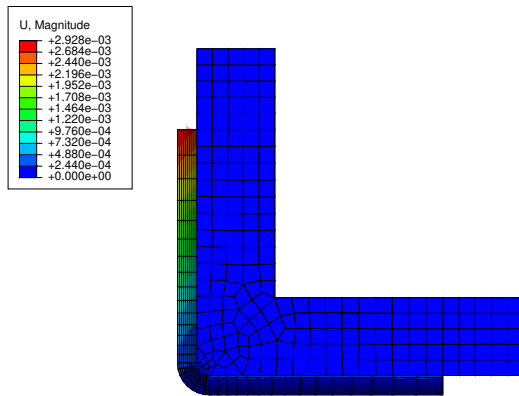
For more information, see *Modeling the Watts test of adhesives*.

2024 FD03

Cure Process Simulation: Prediction of Cure-Induced Spring-In of an Angle Bracket

A new example demonstrates the use of multiscale technology combined with cure modeling in Abaqus/Standard to model the cure-induced spring-in deformation of an angle bracket.

Benefits: The tests demonstrate the composite curing process modeling capability in Abaqus/Standard.



For more information, see *Prediction of cure-induced spring-in of an angle bracket*.

2024 FD01

Failure of Blunt Notched Fiber Metal Laminates

A new test simulates failure and damage in a fiber metal laminate containing a blunt notch subjected to quasi-static loading.

Benefits: The tests demonstrate the capability in Abaqus/Standard. Multiscale modeling is used.

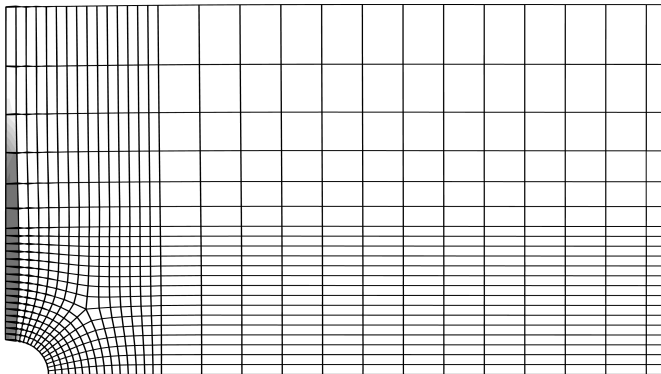


Figure 19: Fiber damage pattern in the 0° fiber-reinforced epoxy layer for the 0° loading direction (using the Hashin damage model, DAMAGEFT contour plot).

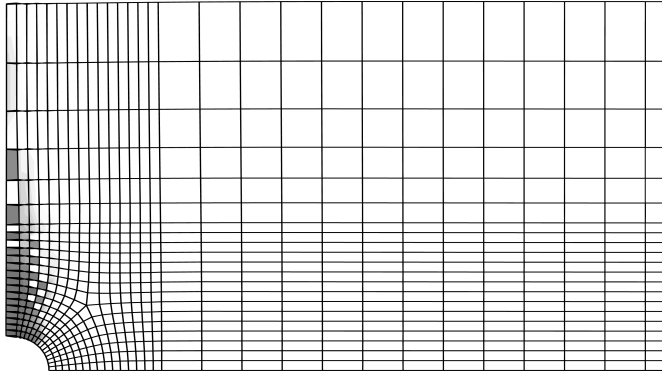


Figure 20: Matrix damage pattern in the 0° fiber-reinforced epoxy layer for the 0° loading direction (using the Hashin damage model, DAMAGEMT contour plot).

For more information, see *Failure of blunt notched fiber metal laminates*.

Execution

This page describes recent changes in Abaqus Execution.

2024 FD04

Optimization and Parallel Execution

For parallel execution using GPGPU acceleration, you now specify the number of GPUs as the total number of GPUs for the analysis rather than specifying the number of GPUs per MPI process.

Benefits: Specifying GPGPU acceleration using the total number of GPUs is more consistent with how you specify the number of CPUs allocated to an analysis and permits greater flexibility for job execution in heterogeneous compute environments.

For more information, see *Abaqus/Standard and Abaqus/Explicit Execution, Optimization Execution, Environment File Settings, Parallel Execution in Abaqus/Standard*, and *Parallel Execution in Abaqus/Explicit*.

2024 FD03

Direct Sparse Solver Support for Steps with Unsymmetric Matrix Storage

For shared memory parallel (SMP) execution of steady-state dynamic direct procedures with unsymmetric matrix storage, Abaqus/Standard now uses the pure thread-based complex solver by default.

Benefits: The pure thread-based complex solver provides a solution that is typically at least twice as fast as the previous default option, the hybrid direct sparse solver.

For more information, see *Parallel Execution in Abaqus/Standard*.

2024 FD02

Execution Procedures User Assistance Reorganized

Content in the Execution Procedures section of the Abaqus Execution Guide is now divided into several subsections.

Benefits: The new organization makes it easier to locate user assistance for the **abaqus** commands you want to run.

Topics are now divided into the following subsections:

- *Abaqus Execution*
- *Abaqus Co-Simulation*
- *Abaqus Utilities*
- *Translating Other Solver Input to Abaqus Input*
- *Translating Abaqus Output to Another Format*

For more information, see *Execution Procedures*.

2024 FD01

Enhanced Translation of LS-DYNA Keyword Files to Abaqus

You can now translate LS-DYNA keyword files that use the `i10` format option in reading integer data.

Benefits: The ability to read these data improves the usability of the **abaqus fromdyna** translator.

You can specify the format for individual keywords by appending the percent sign at the end of the keyword, as shown in the following examples:

- `*ELEMENT_BEAM %`
- `*ELEMENT_SHELL %`
- `*ELEMENT_SOLID %`
- `*NODE %`

In addition, you can specify the format globally for all applicable keywords in the LS-DYNA keyword file, by adding an option to the execution command of the translator or by declaring the following keyword option:

`*KEYWORD i10=y`

For more information, see *Translating LS-DYNA Data Files to Abaqus Input Files*.

Getting Started with Abaqus/CAE

There are no enhancements.

GUI Toolkit

There are no enhancements.

Interactions

This page describes recent changes in Abaqus Interactions.

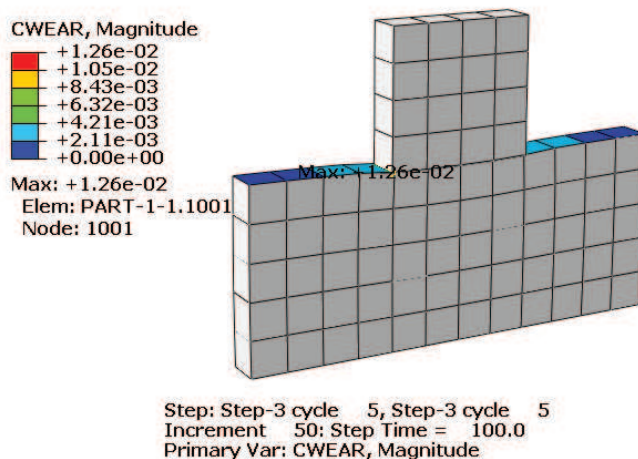
2025 GA

Step Cycling for Contact Wear Simulations

You can now use step cycling and step cycling controls to avoid explicitly specifying each instance of repeated steps associated with wear accumulation and to allow the number of physical wear cycles represented per simulated cycle to evolve based on current wear rates in Abaqus/Standard.

Benefits: Step cycling makes it easier to specify the model and enables more computationally efficient wear simulations.

In many contact wear workflows, surface wear accumulates over thousands of physical wear cycles. It is often prohibitively expensive to simulate each physical wear cycle, so each simulated cycle typically represents a "batch" of multiple similar physical cycles. The animation below shows a wear simulation over five simulated cycles that represent 110,000 physical wear cycles in total. Step cycling allows the "batch size" to evolve during the simulation based on the maximum incremental wear distance in the simulated cycle. The number of simulated cycles is often not known before running such a simulation. For each simulated cycle, the user-specified wear coefficients are scaled by the current batch size.



The animation shows the wear distance accumulation on the slider and the bottom block as the slider repeatedly moves back and forth. In this example, the simulated wear cycle is modeled in two steps:

- In “Step-2” the slider moves to the right and returns to the center.
- In “Step-3” the slider moves to the left and returns to the center.

These two steps are repeated using step cycling until a threshold wear distance exceeds 0.012 or reaches a cap of 10 repetitions of the simulated wear cycle. The batch size for scaling the wear coefficients has an initial value of 10,000 and a cap of 25,000. The target incremental wear distance is 0.006 accumulated per simulated cycle. The batch size for the next simulated wear cycle is internally calculated so that the estimated maximum wear accumulation is less than the target value of 0.006 while respecting the batch size cap. The example simulation

reaches the threshold wear distance in 5 simulated cycles. The batch size for the initial cycle is 10,000 based on user input. For the subsequent cycles, the batch sizes are calculated to be 25,000 (the cap value) while maintaining the maximum wear accumulated in each simulated cycle to be less than 0.006.

For more information, see *Contact Wear*, **STEP CYCLING*, and **STEP CYCLING CONTROL*.

Contact Involving Beams with Noncircular Cross-Sections in Abaqus/Explicit

The contact surface representation of beams with noncircular cross-sections in Abaqus/Explicit now uses internally generated meshes of surface elements that are available for postprocessing.

Benefits: The new contact surface representation is consistent with Abaqus/Standard, and it more accurately and realistically represents contact output.

In earlier releases of Abaqus/Explicit, similar internally generated meshes could be used for simulations but they were not available for postprocessing. The following images compare plots of contact forces and contact opening distances on beam elements to plots on internally generated surface-element representations of the beams. Plots on surface elements show variations over the surfaces, similar to plots that would occur on the outer surface of a solid body, whereas plots based on beam elements are limited in that they represent a single value at each axial location.

Furthermore, some rounding of edges in the contact representation to improve numerics is achieved by offsetting nodes of internally generated surface elements inward and adding a small corresponding contact thickness. This contact thickness is not represented in plots, so the parts appear a bit smaller than the actual size.

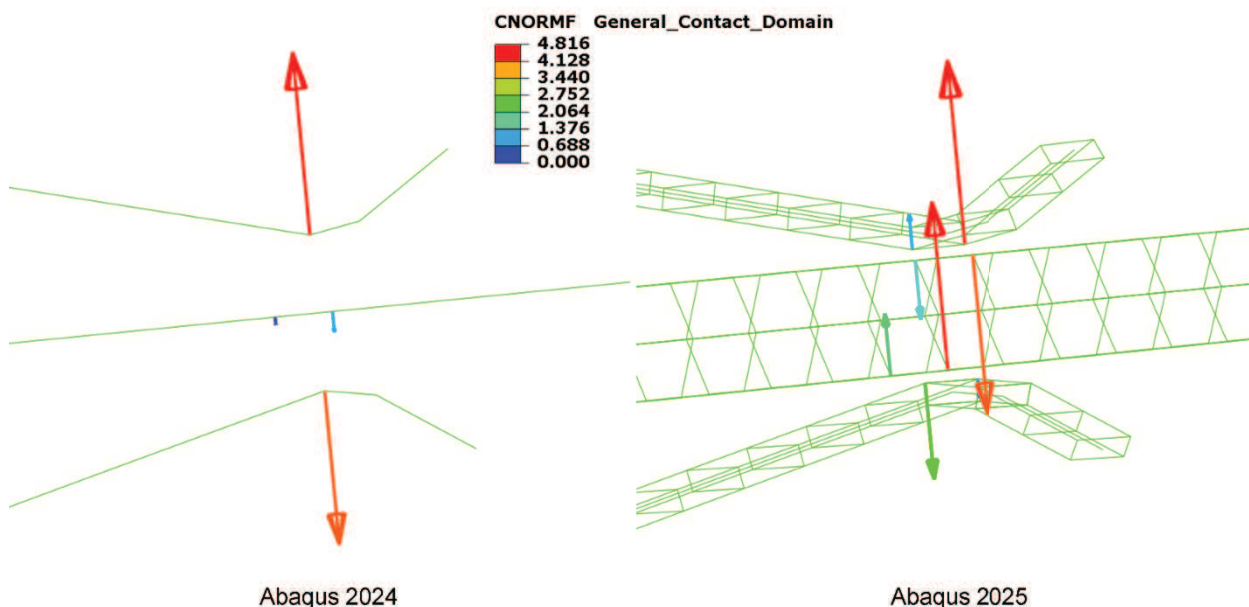


Figure 21: Tweezers beam model contact force vector plot across versions (beam rendering not available for vector plots).

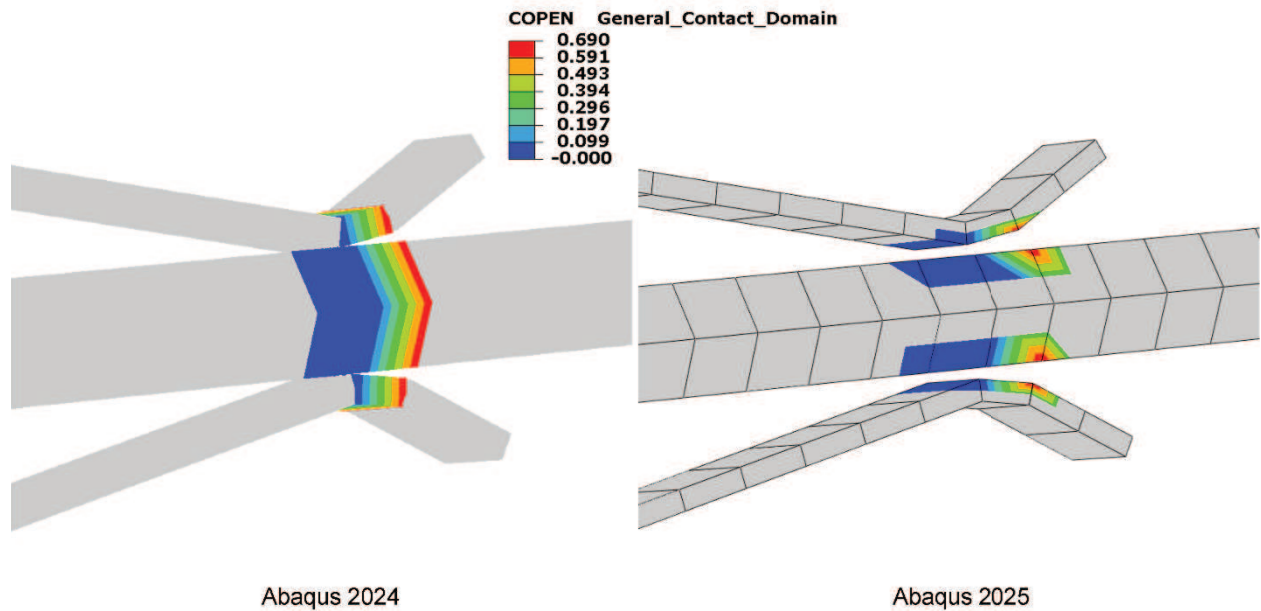


Figure 22: Tweezers beam model contact opening distance output in previous versions (beam rendering shown for Abaqus 2024).

For more information, see *About General Contact in Abaqus/Explicit*, *About General Contact in Abaqus/Standard*, and **CONTACT CONTROLS ASSIGNMENT*.

2024 FD03

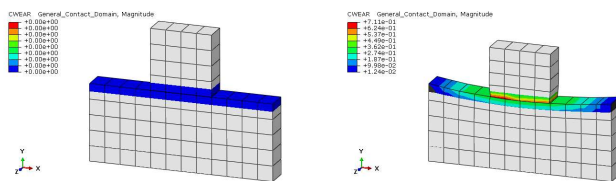
Arbitrary Lagrangian-Eulerian (ALE) Adaptive Meshing Supports Contact Wear in Abaqus/Explicit

You can now use arbitrary Lagrangian-Eulerian (ALE) adaptive meshing together with wear in Abaqus/Explicit, which accounts for local surface wear distances in underlying element calculations.

Benefits: Users can more easily and accurately account for wear distances that are large compared to element dimensions.

The capability to simulate wear in general contact in Abaqus/Explicit was introduced in 2024 FD02. In that release Abaqus/Explicit treated the evolution of wear exclusively by considering nodal wear distances in contact penetration calculations. Now you can use arbitrary Lagrangian-Eulerian (ALE) adaptive meshing together with wear in Abaqus/Explicit to account for surface wear (erosion) in part geometries while preserving mesh quality, even for wear distances on the order of element dimensions.

The simulation on the left below shows the preexisting behavior (wear distances accounted for with offsets in contact penetration calculations). The simulation on the right demonstrates the use of ALE in the context of wear modeling. Similar wear accumulation occurs for both models, but it is much easier to interpret the wear configurations with ALE. Most wear occurs on the smaller block in this model, with the maximum wear distance being approximately half of the original element depth. In this example, ALE mesh smoothing effectively provides a realistic representation of the evolving part geometry while maintaining good mesh quality.



For more information, see *Contact Wear* and **ADAPTIVE MESH*.

2024 FD02

Cohesive Contact in Abaqus/Explicit

Performance for Abaqus/Explicit models with cohesive contact is improved compared to earlier releases. In addition, scaling the default contact penalty stiffness now also scales the default contact cohesive stiffness in Abaqus/Explicit; this behavior is consistent with that in Abaqus/Standard.

Benefits: The improvements facilitate more efficient user workflows and user adjustments to increase or decrease the contact cohesive stiffness from its default value.

Enhancements to cohesive contact in Abaqus/Explicit make cohesive contact a preferred alternative to surface-based tie constraints for modeling permanent, stiff bonds in more cases (for example, models where ties result in large implicit constraint systems or when you require interfacial traction results across connected surfaces).

Performance improvements for cohesive contact are most noticeable for models with a large fraction of nodes participating in cohesive contact. For example, simulations for the models shown in [Figure 3](#) and [Figure 4](#) with permanent, stiff cohesive contact bonds have sped up by 13% and 12%, respectively, compared to simulations in 2024 FD01.

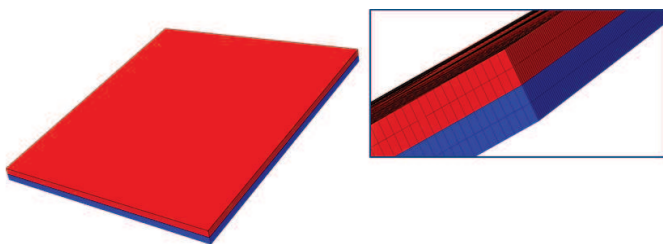


Figure 23: Two plates modeled with solid elements bonded together with permanent, stiff cohesive contact.

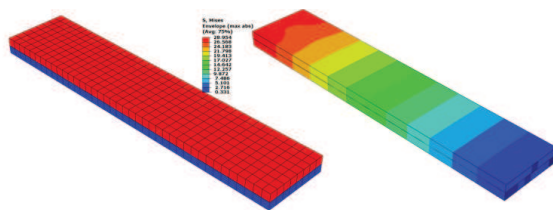


Figure 24: Two plates modeled with shell elements (shown with shell thickness rendering) bonded together with permanent, stiff cohesive contact.

A second enhancement allows the default cohesive stiffness to be scaled by existing options to scale the contact penalty stiffness in Abaqus/Explicit. Cohesive contact, like other contact constraints, has a finite bond stiffness. You can assign the bond stiffness or allow it to take on the same contact penalty stiffness that would be in effect to resist penetrations for regular contact behavior, as discussed in *Contact Cohesive Behavior*. You can use a scale factor to increase the cohesive stiffness relative to the default value.

Figure 5 shows two blocks bonded together with uniform tensile (negative pressure) loading of $1 \times 10^6 \text{ N/m}^2$ applied to the top face of the top block while the bottom of the bottom block is constrained. Loading is smoothly ramped on and damping is included to obtain a quasi-static solution.

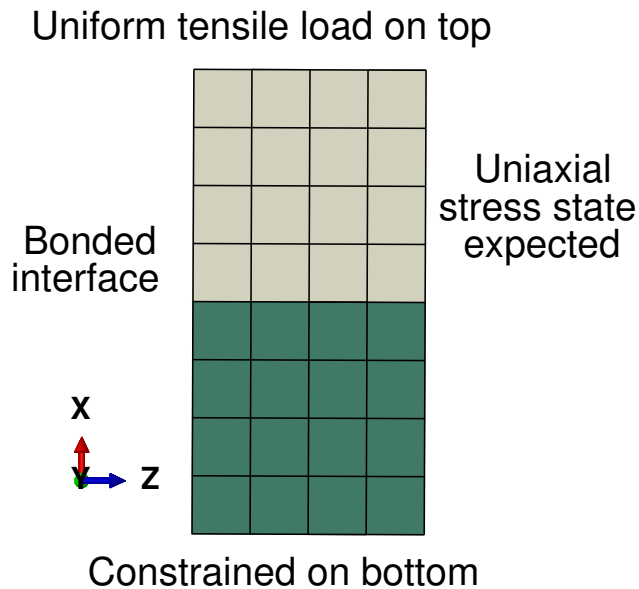


Figure 25: Model characteristics.

Figure 6 shows that the axial stress results correspond to the expected uniaxial stress solution regardless of details of how the bond is modeled.

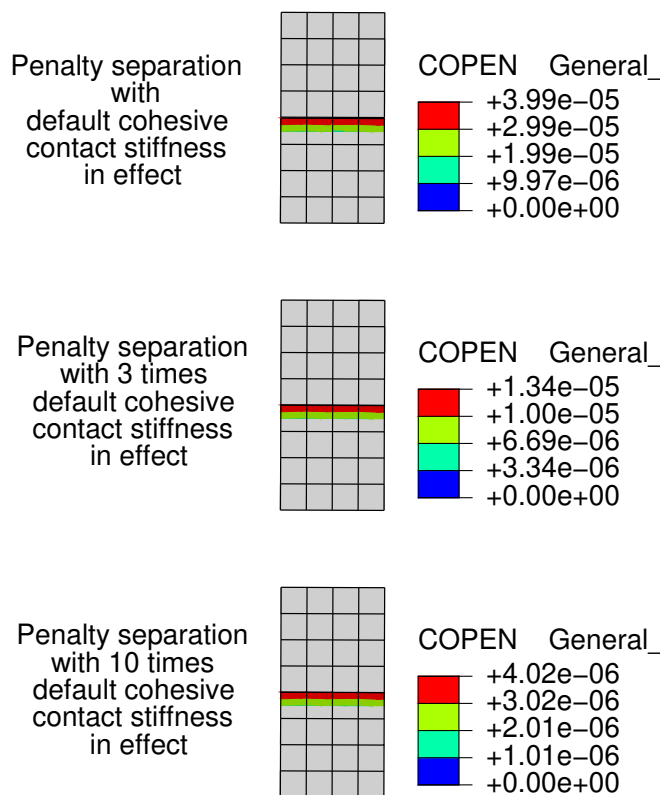


Figure 26: Vertical stress (S11) for three model variants.

Figure 7 shows that some additional displacement occurs when a penalty stiffness is used to enforce the bond. Increasing the penalty stiffness via a scale factor is a convenient mechanism to control the resistance to relative motion across the bond.

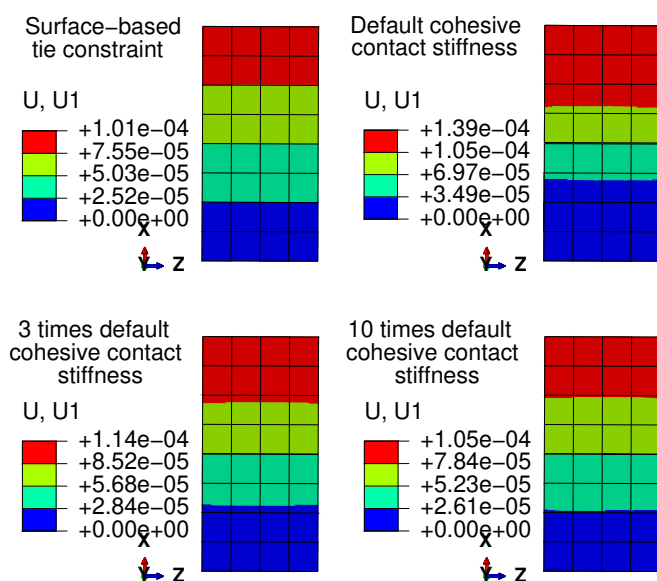


Figure 27: Vertical displacement (U1) for four model variants.

The mesh is quite coarse in this example. The default penalty stiffness would be greater with a more refined mesh. Increasing the penalty stiffness tends to decrease the time increment size in Abaqus/Explicit unless contact mass scaling is used (see *Mass Scaling to Account for Contact Stiffness*).

Alternatively, you can directly specify cohesive stiffness components, as discussed in *Cohesive Stiffness*. When you specify the cohesive stiffness, the penalty stiffness scale factor has no effect.

For more information, see *Contact Cohesive Behavior*, **COHESIVE BEHAVIOR*, **CONTACT CONTROLS ASSIGNMENT*, **CONTACT MASS SCALING*, *Contact Controls for General Contact in Abaqus/Explicit*, and *Mass Scaling*.

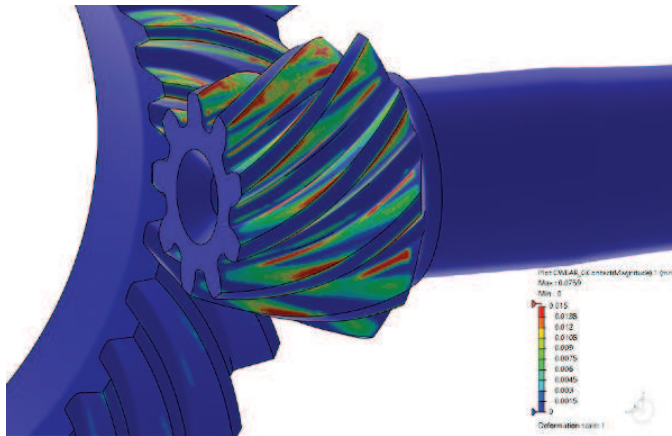
Contact Wear in Abaqus/Explicit

Abaqus/Explicit can now model the evolution of wear and visualize nodal wear distances on contact surfaces due to mechanical contact based on the Archard's wear equation.

Benefits: You can now model wear accumulated at the contact level in Abaqus/Explicit.

This new capability is available in Abaqus/Explicit for general contact analysis involving solid elements. Similar to the implementation introduced for Abaqus/Standard in 2024 FD01, the modeling approach accounts for local surface wear distances in contact penetration calculations but not in underlying element calculations.

The image below shows an example of accumulation of wear (shown as CWEAR contour output) in a simplified helical bevel gear analysis.



For more information, see *Contact Wear*, **SURFACE PROPERTY*, **SURFACE PROPERTY ASSIGNMENT*, and **WEAR SURFACE PROPERTIES*.

Usability enhancement for specifying reference thread geometry in Abaqus/Standard

Abaqus/Standard allows reference thread geometry to be specified on either surface (secondary or main) of a small-sliding interaction for general contact.

Benefits: Reference thread data for reference thread geometry can be specified for either surface of a small-sliding interaction of general contact.

Allowing reference thread geometry to be specified on either surface of a small-sliding interaction for general contact improves usability. Previously, this data was expected to be specified on whichever surface acting as the secondary surface of the contact formulation. However, main and secondary roles are typically left as default

assignments for general contact in Abaqus/Standard, so a user typically would not know the secondary and main roles before running a simulation (and typically should not need to be aware of those roles).

The right side of [Figure 8](#) shows that results in Abaqus 2024 HF2 are insensitive to whether reference thread geometry is specified for the bolt or hole surface for small sliding interactions of general contact. In prior versions, the expected behavior was obtained only if reference thread geometry was specified for the secondary surface.

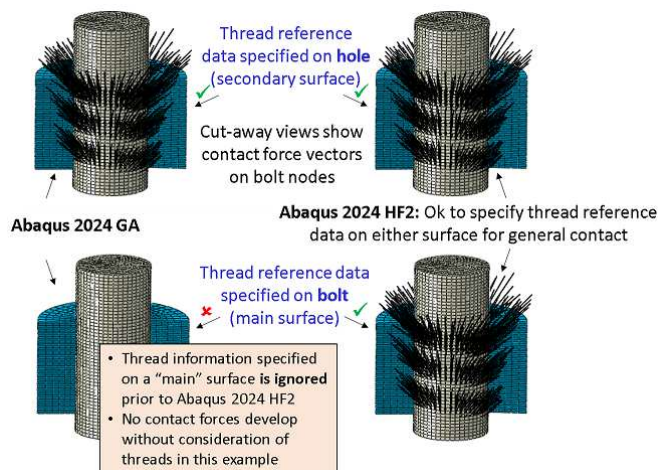


Figure 28: Cut-away views of contact force vectors on bolt nodes.

2024 FD01

Contact Wear in Abaqus/Standard

You can now model the evolution of wear and visualize nodal wear distances on contact surfaces due to mechanical contact, based on the Archard's wear equation.

Benefits: Abaqus/Standard can now handle wear modeling workflows in automotive, aerospace, life sciences, and packaging applications.

In many workflows, mechanical contact can cause wear on contacting surfaces due to mechanisms like abrasion and fretting. The new wear capability in Abaqus/Standard models wear evolution based on the Archard's wear equation. Accordingly, contact nodal wear distances evolve based on wear properties of the surface material and on contact variables such as normal stress and slip distance. Abaqus/Standard treats wear distances as nodal offsets to the contact algorithm, resulting in a simplified approach to model wear fully within contact. It does not model the changes to geometry from wear explicitly. The approach considers the two-way physical influence of wear distance evolution on contact stresses (and underlying material stresses) and vice versa.

[Figure 9](#) shows two blocks in contact.

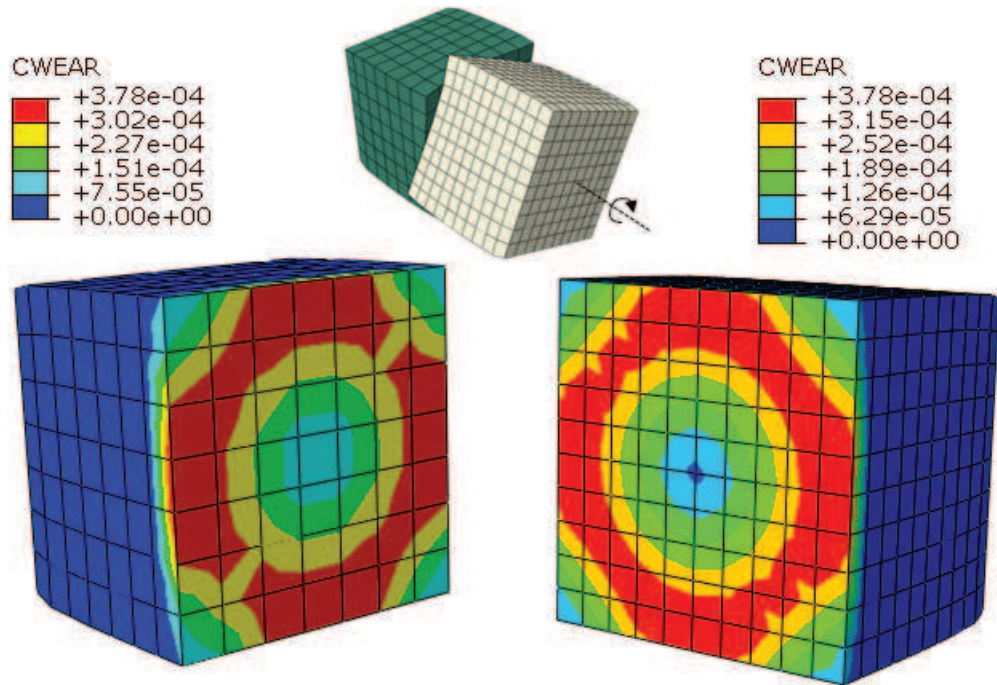


Figure 29: Wear accumulation on contact surfaces.

Wear properties for the Archard's wear equation are specified on the contact surfaces. Nodal wear distances accumulate as the blocks slide relative to each other under compression. The nodal wear distances on the two surfaces are shown as CWEAR output contours at the end of the simulation.

For more information, see *Contact Wear*, *Contact wear modeling*, **SURFACE PROPERTY*, **SURFACE PROPERTY ASSIGNMENT*, and **WEAR SURFACE PROPERTIES*.

Keywords

This page describes recent changes in Abaqus Keywords.

2025 GA

*FLUID EXCHANGE ACTIVATION

The Interaction module in Abaqus/CAE now supports this option.

Benefits: This feature extends the capabilities of Abaqus/CAE.

For more information, see **FLUID EXCHANGE ACTIVATION*.

*STEP CYCLING (new)

Specify a step or series of steps to be repeated over a set number of cycles.

Benefits: This feature expands the capabilities of Abaqus/Standard.

For more information, see **STEP CYCLING*.

*STEP CYCLING CONTROL (new)

Specify a sensor-based or a contact wear-based criterion to trigger early termination of step cycling before a set number of step cycles are complete.

Benefits: This feature expands the capabilities of Abaqus/Standard.

For more information, see **STEP CYCLING CONTROL*.

2024 FD04

*BEAM GENERAL SECTION

Use the new optional CONTROLS parameter to specify bulk viscosity scale factors or to ramp up the initial stress in an Abaqus/Explicit analysis.

Benefits: This feature expands the capabilities of Abaqus/Explicit.

For more information, see **BEAM GENERAL SECTION*.

*BEAM SECTION

Use the new optional CONTROLS parameter to specify bulk viscosity scale factors or to ramp up the initial stress in an Abaqus/Explicit analysis.

Benefits: This feature expands the capabilities of Abaqus/Explicit.

For more information, see **BEAM SECTION*.

*PLASTICITY CORRECTION

The Property module in Abaqus/CAE now supports this option.

Benefits: This feature extends the capabilities of Abaqus/CAE.

For more information, see **PLASTICITY CORRECTION*.

2024 FD03

*CONTROLS

You can use the new CONVERGENCE CHECK parameter to specify which convergence conditions Abaqus/Standard must satisfy before it accepts the incremental solution as converged.

Benefits: This feature expands the capabilities of Abaqus/Standard.

For more information, see **CONTROLS*.

*ELECTRICAL CONDUCTIVITY

You can now use this option to define electrical conductivity for elements that support the electric potential degree of freedom in a coupled thermal-electrochemical, coupled thermal-electrochemical-structural, or coupled thermal-electrochemical-structural-pore pressure analysis.

Benefits: This feature expands the capabilities of Abaqus/Standard.

In earlier releases, you could use **ELECTRICAL CONDUCTIVITY* only in coupled thermal-electrical and coupled thermal-electrical-structural analyses.

For more information, see **ELECTRICAL CONDUCTIVITY*.

*ELECTRICAL RESISTIVITY (new)

Define electrical resistivity for elements that support the electric potential degree of freedom in a coupled thermal-electrical, coupled thermal-electrical-structural, coupled thermal-electrochemical, coupled thermal-electrochemical-structural, or coupled thermal-electrochemical-structural-pore pressure analysis.

Benefits: This feature expands the capabilities of Abaqus/Standard.

You can use the new option to define electrical resistivity (as opposed to conductivity). This method of specifying electrical resistivity is required for piezoresistive materials.

For more information, see **ELECTRICAL RESISTIVITY* and *Piezoresistive Effect*.

*ELECTRIC MACHINE LOAD (new)

Define electromagnetic force loads for an electric machine.

Benefits: This feature expands the capabilities of Abaqus/Standard.

For more information, see **ELECTRIC MACHINE LOAD*.

*ELECTRIC MACHINE PROPERTY (new)

Define electric machine properties (or parameters) that are needed to apply electric machine loads.

Benefits: This feature expands the capabilities of Abaqus/Standard.

For more information, see **ELECTRIC MACHINE PROPERTY*.

***ELEMENT USER OUTPUT VARIABLES (new)**

Specify the number of whole element and integration point user-defined variables defined for each element in user subroutine UVAREL.

Benefits: This feature expands the capabilities of Abaqus/Standard.

For more information, see **ELEMENT USER OUTPUT VARIABLES*.

***EXPANSION**

You can use the new DEFINITION parameter to specify secant or tangent thermal expansion coefficients.

Benefits: This feature expands the capabilities of Abaqus/Standard.

For more information, see **EXPANSION*.

***MODAL DYNAMIC**

You can use the new SOLVER parameter to specify if you want to use the state space solver or the conventional solver.

Benefits: This feature expands the capabilities of Abaqus/Standard.

For more information, see **MODAL DYNAMIC*.

***PIEZORESISTIVITY (new)**

Define the piezoresistivity coefficients for a piezoresistive material.

Benefits: This feature expands the capabilities of Abaqus/Standard.

For more information, see **PIEZORESISTIVITY*.

***SECTION CONTROLS**

You can now use this option to control the activation of bending stabilization for full-integration linear elements modeled with hyperelastic materials and to specify the scale factor.

Benefits: This feature expands the capabilities of Abaqus/Standard and Abaqus/Explicit.

For more information, see **SECTION CONTROLS*.

***UNIT SYSTEM**

You can use the new SYSTEM parameter to specify the unit system used in the model.

Benefits: This feature expands the capabilities of Abaqus/Standard and Abaqus/Explicit.

For more information, see **UNIT SYSTEM*.

***VISCOELASTIC**

You now set the LAW parameter equal to MODIFIED DARVEAUX to choose a modified Darveaux law for a nonlinear viscoelastic material definition.

Benefits: This feature expands the capabilities of Abaqus/Standard and Abaqus/Explicit.

For more information, see **VISCOELASTIC*.

2024 FD02

***CONTOUR INTEGRAL**

You can use the LENGTH SCALING FACTOR parameter to indicate that Abaqus/Standard will not adjust the locations or the radii of all nodes along the crack front when evaluating the line integral.

Benefits: This feature expands the capabilities of Abaqus/Standard.

For more information, see **CONTOUR INTEGRAL*.

***CURE GLASS TRANSITION TEMPERATURE**

You can now use this option with all cure kinetics models. In earlier releases, you could use it only with the Grindling model.

Benefits: This feature expands the capabilities of Abaqus/Standard.

For more information, see **CURE GLASS TRANSITION TEMPERATURE*.

***DENSITY**

Abaqus/Explicit now supports the PORE FLUID parameter for undrained pore fluid flow and stress analysis.

Benefits: This feature expands the capabilities of Abaqus/Explicit.

For more information, see **DENSITY*.

***DYNAMIC**

You can now set the PORE PRESSURE parameter equal to UNDRAINED to specify that an undrained pore fluid flow and stress analysis is to be performed in Abaqus/Explicit.

Benefits: This feature expands the capabilities of Abaqus/Explicit.

For more information, see **DYNAMIC*.

***EXPANSION**

Abaqus/Explicit now supports the PORE FLUID parameter for undrained pore fluid flow and stress analysis.

Benefits: This feature expands the capabilities of Abaqus/Explicit.

For more information, see **EXPANSION*.

***HYPERELASTIC**

You can use the new HENCKY parameter to use the Hencky model in Abaqus/Standard.

Benefits: This feature expands the capabilities of Abaqus/Standard.

For more information, see **HYPERELASTIC*.

***IMPORT**

You can use the full-model input format to specify that all import model data, including elements and nodes, element sets and node sets, sections and materials, etc., are given in the input.

Benefits: This feature expands the capabilities of Abaqus/Standard and Abaqus/Explicit.

For more information, see **IMPORT*.

***IMPORT CONTROLS**

You can set the new MODEL parameter equal to YES to indicate that you want to use the full-model input format. In an Abaqus/Explicit analysis, you can use the new TIE CONFIG parameter to indicate whether to use the original or updated configuration in the tie constraint formation for all surfaces of the imported model.

Benefits: This feature expands the capabilities of Abaqus/Standard and Abaqus/Explicit.

For more information, see **IMPORT CONTROLS*.

***INITIAL CONDITIONS**

You can now set the TYPE parameter equal to PORE PRESSURE to define the initial pore pressure for undrained pore fluid flow and stress analysis in Abaqus/Explicit. Similarly, you can set the TYPE parameter equal to POROSITY to define the initial porosity values and to RATIO to define the initial void ratio values.

Benefits: This feature expands the capabilities of Abaqus/Explicit.

For more information, see **INITIAL CONDITIONS*.

***POROUS BULK MODULI**

You can now use this option in Abaqus/Explicit to define the porous bulk moduli for undrained pore fluid flow and stress analysis.

Benefits: This feature expands the capabilities of Abaqus/Explicit.

For more information, see **POROUS BULK MODULI*.

***SURFACE PROPERTY**

You can now use this option in Abaqus/Explicit to define named surface properties for contact-based wear modeling.

Benefits: This feature expands the capabilities of Abaqus/Explicit.

For more information, see **SURFACE PROPERTY*.

***VISCOELASTIC**

You now set the LAW parameter equal to DARVEAUX to choose a Darveaux law.

Benefits: This feature expands the capabilities of Abaqus/Standard and Abaqus/Explicit.

For more information, see **VISCOELASTIC*.

***WEAR SURFACE PROPERTIES**

You can use this option in Abaqus/Explicit to specify wear properties to activate wear distance accumulation at contact surface nodes.

Benefits: This feature expands the capabilities of Abaqus/Explicit.

For more information, see **WEAR SURFACE PROPERTIES*.

2024 FD01

***ALLOWABLE STRESS (new)**

Specify the allowable stress of a composite at the initiation of damage in a multiscale material.

Benefits: This feature expands the capabilities of Abaqus/Standard.

For more information, see **ALLOWABLE STRESS*.

***CFILM**

You can use the new SINK AMPLITUDE parameter to define the variation of the sink temperature with time. The SINK AMPLITUDE parameter supersedes the AMPLITUDE parameter used in earlier versions.

Benefits: This feature expands the capabilities of Abaqus/Standard and Abaqus/Explicit.

For more information, see **CFILM* and *Thermal Loads*.

***CONNECTOR SECTION**

For SLIPRING connections, you can now request that Abaqus/Standard computes the contact angle automatically during the analysis.

Benefits: This feature expands the capabilities of Abaqus/Standard.

For more information, see **CONNECTOR SECTION*.

***CONTOUR INTEGRAL**

You can now use the SURFACE NORMAL parameter when tetrahedral elements are used for fracture mechanics studies based on the conventional finite element method.

Benefits: This feature expands the capabilities of Abaqus/Standard.

For more information, see **CONTOUR INTEGRAL*.

*CRADIATE

You can use the new AMBIENT AMPLITUDE parameter to define the variation of the ambient temperature with time. The AMBIENT AMPLITUDE parameter supersedes the AMPLITUDE parameter used in earlier versions. You can also use the new RADIATION AMPLITUDE parameter to define a variation of the radiation flux with time.

Benefits: This feature expands the capabilities of Abaqus/Standard and Abaqus/Explicit.

For more information, see **CRADIATE* and *Thermal Loads*.

*DAMAGE EVOLUTION

You can now set the TYPE parameter equal to DISCONTINUOUS to define the residual stiffness scaling factor after the initiation of damage.

Benefits: This feature expands the capabilities of Abaqus/Standard and Abaqus/Explicit.

For more information, see **DAMAGE EVOLUTION*.

*DAMAGE INITIATION

You can now define general stress/strain-based damage initiation as a material property for continuum damage modeling in anisotropic materials. You can also now define damage initiation as a material property for continuum damage modeling in fiber-reinforced composites using the multiscale material model in Abaqus/Standard.

Benefits: This feature expands the capabilities of Abaqus/Standard and Abaqus/Explicit.

For more information, see **DAMAGE INITIATION*.

*DAMPING

Abaqus/Standard now supports the BAND LIMITED, HIGH FREQUENCY CUTOFF, and LOW FREQUENCY CUTOFF parameters.

Benefits: This feature expands the capabilities of Abaqus/Standard.

For more information, see **DAMPING*.

*DISTRIBUTING

You can use the new SINGLE FACET parameter to specify whether or not to automatically convert distributing couplings where all cloud nodes are on a single shell facet to a constraint in Abaqus/Explicit.

Benefits: This feature expands the capabilities of Abaqus/Explicit.

For more information, see **DISTRIBUTING*.

*EXPANSION

In an Abaqus/Standard analysis, you can now use the PORE FLUID parameter to define field expansion of the pore fluid in a porous medium. Similarly, the USER and ZERO parameters now take field expansion into account.

Benefits: This feature expands the capabilities of Abaqus/Standard.

For more information, see **EXPANSION*.

*FILM

You can use the new SINK AMPLITUDE parameter to define the variation of the sink temperature with time. The SINK AMPLITUDE parameter supersedes the AMPLITUDE parameter used in earlier versions.

Benefits: This feature expands the capabilities of Abaqus/Standard and Abaqus/Explicit.

For more information, see **FILM* and *Thermal Loads*.

*NSET

You can now use the new SURFACE parameter to generate a node set from a previously defined surface.

Benefits: This feature expands the capabilities of Abaqus/Standard and Abaqus/Explicit.

This enhancement makes it easier to create a node set from a surface definition. All the nodes included in the surface are assigned to the specified node set.

For more information, see **NSET*.

*RADIATE

You can use the new AMBIENT AMPLITUDE parameter to define the variation of the ambient temperature with time. The AMBIENT AMPLITUDE parameter supersedes the AMPLITUDE parameter used in earlier versions. You can also use the new RADIATION AMPLITUDE parameter to define a variation of the radiation flux with time.

Benefits: This feature expands the capabilities of Abaqus/Standard and Abaqus/Explicit.

For more information, see **RADIATE* and *Thermal Loads*.

*REDUCED BASIS GENERATE (new)

Indicate that the step should be analyzed as a reduced basis generation step.

Benefits: This feature expands the capabilities of Abaqus/Standard.

For more information, see **REDUCED BASIS GENERATE*.

*SFILM

You can use the new SINK AMPLITUDE parameter to define the variation of the sink temperature with time. The SINK AMPLITUDE parameter supersedes the AMPLITUDE parameter used in earlier versions.

Benefits: This feature expands the capabilities of Abaqus/Standard and Abaqus/Explicit.

For more information, see **SFILM* and *Thermal Loads*.

*SRADIATE

You can use the new AMBIENT AMPLITUDE parameter to define the variation of the ambient temperature with time. The AMBIENT AMPLITUDE parameter supersedes the AMPLITUDE parameter used in earlier versions. You can also use the new RADIATION AMPLITUDE parameter to define a variation of the radiation flux with time.

Benefits: This feature expands the capabilities of Abaqus/Standard and Abaqus/Explicit.

For more information, see **SRADIATE* and *Thermal Loads*.

*STEADY STATE DYNAMICS

You can use the new MODE BASED parameter to define the conventional modal solver or the spectral modal solver that can perform better than the conventional solver in analyses with many modes, frequency points, and load cases for finite element models with special, but commonly used, damping properties.

Benefits: This feature expands the capabilities of Abaqus/Standard.

For more information, see **STEADY STATE DYNAMICS* and *Mode-Based Steady-State Dynamic Analysis*.

*SUBMODEL CUT (new)

Identify a submodel cut on which to impose the submodel conditions and specify the submodel control settings specific to a submodel cut.

Benefits: This feature expands the capabilities of Abaqus/Standard.

For more information, see **SUBMODEL CUT*.

*SURFACE PROPERTY

You can now use this option to define named surface properties for contact-based wear modeling.

Benefits: This feature expands the capabilities of Abaqus/Standard.

For more information, see **SURFACE PROPERTY*.

*WEAR SURFACE PROPERTIES (new)

Specify wear properties to activate wear distance accumulation at contact surface nodes.

Benefits: This feature expands the capabilities of Abaqus/Standard.

For more information, see **WEAR SURFACE PROPERTIES*.

2024 GA

*BEAM SECTION

The Property module in Abaqus/CAE now supports this option.

Benefits: This feature extends the capabilities of Abaqus/CAE.

For more information, see **BEAM SECTION*, *Using a Beam Section Integrated during the Analysis to Define the Section Behavior*, and *BeamSection* object.

*CONTACT MASS SCALING

The Interaction module in Abaqus/CAE now supports this option.

Benefits: This feature extends the capabilities of Abaqus/CAE.

For more information, see **CONTACT MASS SCALING* and *Defining contact mass scaling*.

*IMPERFECTION

The Interaction module in Abaqus/CAE now supports this option.

Benefits: This feature extends the capabilities of Abaqus/CAE.

For more information, see **IMPERFECTION*.

*NODE OUTPUT

You can use the new SURFACE parameter to specify the name of the surface for which to request nodal output.

Benefits: This feature makes it easier to request nodal output on a surface in Abaqus/Standard and Abaqus/Explicit. The SURFACE parameter is available when the *NODE OUTPUT option is used in conjunction with the *OUTPUT, HISTORY option for Abaqus/Standard and Abaqus/Explicit or when the *NODE OUTPUT option is used in conjunction with the *OUTPUT, FIELD option for Abaqus/Explicit.

For more information, see **NODE OUTPUT* and **OUTPUT*.

*NODE RESPONSE

You can use the new RANGE OPERATOR parameter to define the operator used for aggregating the sensitivities and design response values over the specified frequency range for the node responses in direct steady-state dynamic adjoint sensitivity analysis.

Benefits: This parameter allows you to control the calculation of the design response over a frequency range for the newly supported steady-state dynamic adjoint sensitivity analysis technique.

By default, Abaqus/Standard outputs the maximum of the design response values as the design response over the frequency range specified for the node response in direct steady-state dynamic analysis when you request adjoint sensitivity computation. Alternatively, you can request output of the sum or average of the node response values over the specified frequency range for the response.

For more information, see **NODE RESPONSE* and *Adjoint Design Sensitivity Analysis*.

*OPERATOR OUTPUT

You can now use the *OPERATOR OUTPUT option in an uncoupled heat transfer analysis. In addition, the LOAD parameter can now take the value NET to output the net flux vector in an uncoupled heat transfer analysis.

Benefits: This feature expands the capabilities of Abaqus/Standard.

For more information, see **OPERATOR OUTPUT*.

Materials

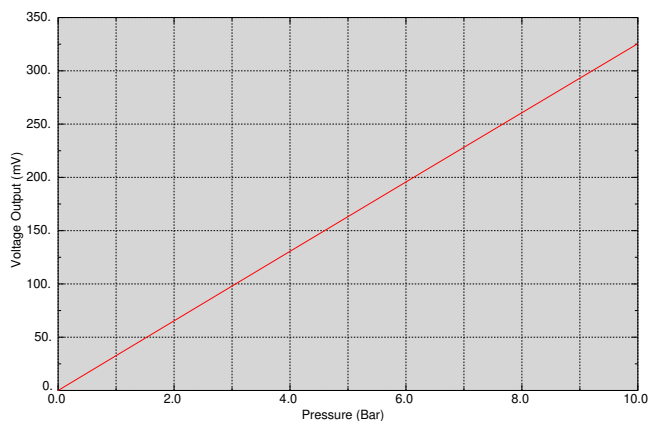
This page describes recent changes in Abaqus Materials.

2024 FD03

Piezoresistive Effect

You can now model materials exhibiting piezoresistive behavior in Abaqus/Standard.

Benefits: You can now analyze and design piezoresistive sensors such as silicon piezoresistive pressure sensors. A piezoresistive material changes its electrical resistivity when stressed. This coupling between stress and electrical resistivity is the material's piezoresistivity property. Silicon piezoresistive sensors have been widely used in the field of microelectromechanical systems. The figure below shows the simulated results between pressure and voltage output of a silicon piezoresistive pressure sensor.



For more information, see *Piezoresistive Effect*.

Electrical Resistivity

You can now define electrical resistivity directly in Abaqus/Standard.

Benefits: This provides an easier and more robust way to define electrical resistivity.

In earlier releases, you could define only electrical conductivity in coupled thermal-electrical elements and coupled thermal-electrical-structural elements. Calculating electrical resistivity from electrical conductivity requires an inversion and might introduce complications for temperature/field variable-dependent properties. Now you can define electrical resistivity directly. Electrical properties must be defined in terms of resistivity for materials with piezoresistive behavior.

For more information, see *Electrical Resistivity*.

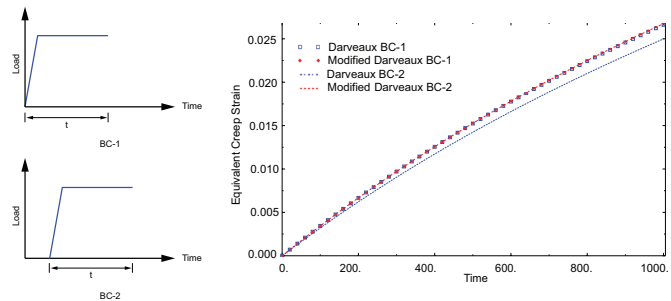
Modified Darveaux Viscous Behavior for Parallel Rheological Framework

You can now use the modified Darveaux model to define creep response in the parallel rheological framework (PRF).

Benefits: The modified Darveaux model provides more accurate results when the step contains idle time or periods of zero loading.

The Darveaux creep law previously available with the PRF material model introduces an explicit dependency of creep strain rate on total time, which can lead to unexpected results when the load is not applied at time zero. In the modified Darveaux model, the creep evolution law is revised by eliminating total time and using the accumulated total steady-state creep strain instead. This approach provides a solution that does not depend on the loading sequence when the step time has an idle or no-load period. Both models provide the same response under constant load and temperature applied at time zero.

To illustrate the difference between the original and modified Darveaux creep law, the following example considers uniaxial loading of a specimen fixed at one end and with a load applied at the other end. The load is ramped up rapidly and then kept constant. Two different loading cases are considered. For BC-1, the load is applied instantaneously at the first increment; for BC-2, the load is applied after some time with no loading. In both cases, the load is kept constant for the same amount of time. The results are compared below for both the original and modified Darveaux laws. For easier comparison, the results obtained from BC-2 are superimposed on BC-1 after shifting the time axis by the initial period with no load.



The figure illustrates that when the load is applied at the very first increment (BC-1), both the original and modified Darveaux models produce the same time history of equivalent creep strain. However, the original Darveaux model displays a different evolution of equivalent creep strain when the load is applied after a certain amount of time (BC-2). By contrast, the modified Darveaux model produces the same equivalent creep strain for both BC-1 and BC-2. This behavior is the expected result because both BC-1 and BC-2 have the same physical loading time.

For more information, see *Parallel Rheological Framework*.

Unsymmetric Solution Scheme for the Superelastic Model

Abaqus/Standard now automatically activates the unsymmetric solution scheme for the superelastic material model if you use the nonassociated flow rule.

Benefits: This capability improves convergence of the superelastic model with the nonassociated flow rule.

For more information, see *Superelasticity*.

Tangent Thermal Expansion

You can now define thermal expansion effects by specifying tangent thermal expansion coefficients.

Benefits: This capability provides flexibility in defining thermal expansion effects and eliminates the need to convert tangent thermal expansion coefficients when tangent coefficients are available.

In earlier releases, Abaqus/Standard required that the thermal expansion coefficients be defined in secant (or total) form. The total form is generally recommended because it leads to thermal strain results that are not dependent on the temperature and field variable history or sensitive to the time increment size. However, when a material undergoes an irreversible transformation (such as during the curing process of polymers), the evolution of thermal strains is better characterized in terms of tangent (or differential) expansion coefficients that are a function of the degree of transformation. In these cases, it is generally not possible to convert the tangent form to an equivalent secant form of thermal expansion, and the only solution was to use user subroutine UEXPAN. Now you can define thermal expansion behavior directly in tangent form in Abaqus/Standard. As in the case of secant thermal expansion, the tangent thermal expansion can be isotropic, transversely isotropic, orthotropic, and fully anisotropic.

Both the secant and the tangent methods produce identical results if the coefficients are constant. If the expansion coefficients depend only on temperature, the conversion from tangent to secant form is always possible; therefore, we recommend that you use the secant form to eliminate the sensitivity of thermal strain results on time increment size. However, in situations when the coefficients depend on field variables, the two methods are not equivalent in general, and it is not possible to perform the conversion. For these situations you can now use the tangent definition.

To illustrate the benefits of the new functionality, consider an example of resin curing. During the curing process, the resin material undergoes two transformations. The first one is a transformation from a liquid to a solid phase, known as gelation. At this point, the material is in a rubbery state. As the glass transition temperature, T_g , increases and eventually exceeds the temperature of the curing process, the material transforms from the rubbery state to the glassy state. Assume that the tangent thermal expansion coefficients for the liquid, rubbery, and glass phases are given by l , r , and g , respectively. Using a value of degree of cure (DOC) of 0.5 as a measure of the gelation transformation from liquid to rubbery state, we can express the tangent thermal expansion coefficient as:

- $= ^l$ if $\text{DOC} \leq 0.5$,
- $= ^r$ if $\text{DOC} > 0.5$ and $(T_g - T) \leq 0$,
- $= ^g$ if $\text{DOC} > 0.5$ and $(T_g - T) > 0$.

You can now use the tangent thermal expansion definition to define these thermal expansion coefficients in tabular form, as functions of two field variables associated with the degree of cure (DOC) and the difference between the glass transition temperature and temperature $\text{TGTDIFF} = (T_g - T)$ as shown below (see also *Prediction of cure-induced spring-in of an angle bracket*).

```
*EXPANSION, DEFINITION=TANGENT, DEPENDENCIES=2
  ^l, , 0.50, 0.0
  ^r, , 0.51, 0.0
  ^l, , 0.50, 0.1
  ^g, , 0.51, 0.1
*USER DEFINED FIELD, TYPE=SPECIFIED
1, DOC
2, TGTDIFF
```

For more information, see *Thermal Expansion*.

Using Low-Density Foam with Eulerian Elements and ALE Analysis

You can now perform a coupled Eulerian-Lagrangian analysis or an analysis involving ALE adaptive meshing for a model with a low-density foam material.

Benefits: You can complete more effective simulations for analyses that involve extreme deformation or distortion of the mesh.

For analyses that involve extreme mesh deformation, coupled Eulerian-Lagrangian analyses are more effective than traditional Lagrangian analyses because the elements do not deform and materials are allowed to flow through the element faces. For an analysis involving ALE adaptive meshing, allowing the mesh to move independently of the material enables you to maintain a high-quality mesh throughout an analysis, even when large deformation or loss of material occurs.

2024 FD02

Nonlinear Viscoelastic Shear Behavior for EOS Materials

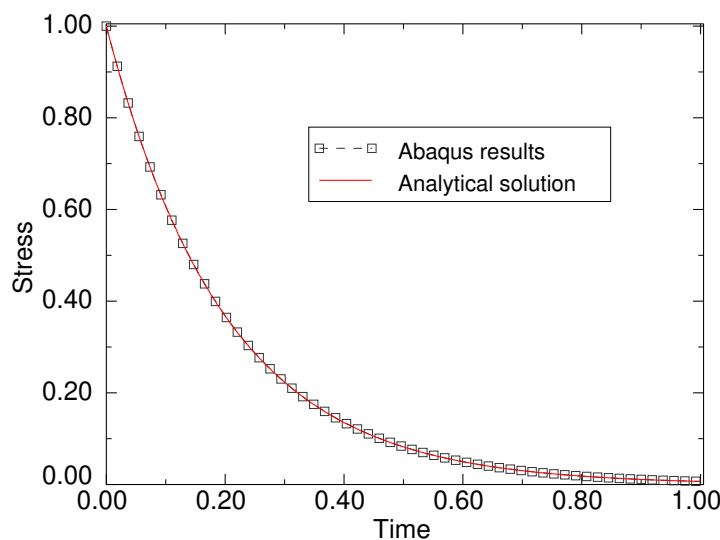
You can now combine nonlinear viscoelastic shear behavior with equation of state (EOS) materials in Abaqus/Explicit to model the response of viscoelastic fluids.

Benefits: The new material capability lets you model viscoelastic fluids and improves the stable time increment when you model fluids with large viscosities.

The family of EOS materials in Abaqus/Explicit now allows you to define nonlinear viscoelastic shear behavior based on a Maxwell-type model (that is, elastic and viscous effects acting in series). In earlier releases, you could define the deviatoric response of EOS materials using only elastic or viscous shear behavior.

The new functionality also provides an efficient method to model viscous fluids with very high viscosity. As the viscosity of a fluid increases, it leads to a significant reduction in the stable time increment required for an explicit dynamic simulation. You can use the new nonlinear viscoelastic behavior to approximate a purely viscous shear response by treating the shear modulus, μ , as a penalty stiffness that you can adjust to achieve a reasonable stable time increment without significant influence on the overall shear viscous response.

The figure below shows the history output of shear stress in a simple shear stress relaxation test with the new material model (dashed line with symbols). The plot compares well with the analytical solution (solid line).



For more information, see *Deviatoric Behavior*.

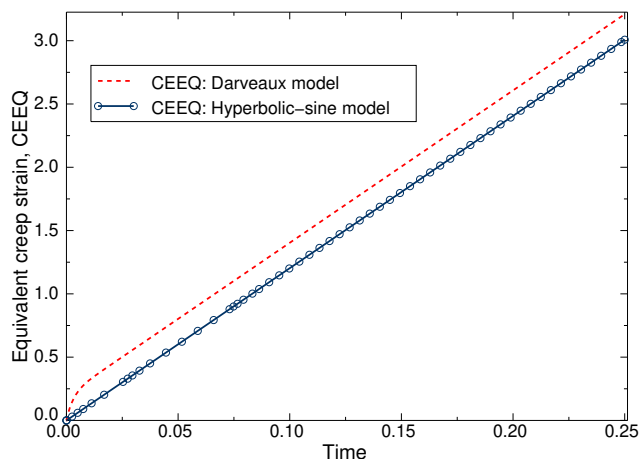
Darveaux Viscous Behavior for Parallel Rheological Framework

You can now use the Darveaux model to define creep response in the parallel rheological framework.

Benefits: The Darveaux model allows you to consider both primary and secondary creep, which enables more accurate results.

In earlier releases, the Darveaux creep model was available in Abaqus/Standard to define creep behavior with linear elasticity as well as the viscous behavior for a two-layer viscoplastic material. This creep model can be useful for capturing rate-dependent behavior in certain classes of polymer such as Nylon 12. You can now use this creep model to define the viscous behavior in models defined using the parallel rheological framework in both Abaqus/Standard and Abaqus/Explicit. The model takes into account both primary (transient) and secondary (steady-state) creep. The secondary creep in the Darveaux model is described by the hyperbolic-sine law, which is modified to include the primary creep contribution that occurs immediately after the stress is applied and before a steady-state creep stage is reached.

To illustrate the difference between the hyperbolic-sine and the Darveaux models, the following example considers a specimen that is fixed at one end and to which a load is applied at the other end. The load is applied rapidly and then kept constant. The specimen's elastic response is modeled using a Hencky hyperelastic material, and the creep behavior is described either by the Darveaux or by the hyperbolic-sine creep model. The results are compared below.



The hyperbolic-sine law model predicts a response that shows the constant equivalent creep strain rate, which often accurately captures the steady-state response at a constant load accurately. The response of the Darveaux model includes the transient creep effects, where the equivalent creep strain rate increases very quickly, stabilizes when the steady-state creep stage is reached, and then remains constant (as in the case of the hyperbolic-sine law).

For more information, see *Parallel Rheological Framework*.

Hencky Hyperelastic Model

You can now use the Hencky model to define hyperelastic material response accurately.

Benefits: You can now accurately predict elastic material response for moderately large deformations.

The Hencky model is an isotropic hyperelastic model that assumes a linear relation between the Kirchhoff stress tensor and the Hencky or logarithmic strain tensor. It is a generalization of the classical Hooke's law for isotropic infinitesimal elasticity. You define the model by providing two material parameters that can be measured at

infinitesimal strains. The Hencky model can produce results that show good agreement with experimental data for a wide class of materials for moderately large deformations. You can also use the model as a finite strain alternative to the isotropic linear elastic model in situations where the isotropic linear elastic model is known to perform poorly (for example, for materials subjected to cyclic loads). In this case, the isotropic linear elastic material, which uses a rate formulation, might produce residual stresses when it returns to the initial (stress-free) configuration. The Hencky model does not show this undesirable behavior because it uses the total deformation to compute stresses.

Figure 1 shows a specimen subjected to simple shear cycling deformations with the mechanical response modeled using the Hencky and isotropic linear elastic models. Twenty cycles are performed with a different number of increments per cycle, and *Figure 2* shows the Mises equivalent stresses. The results for the isotropic linear elastic material accumulate significant errors for a small number of increments per cycle, resulting in large residual stresses after a few cycles. They improve for larger numbers of increments per cycle and stabilize when the number increases to about 100. In contrast, the Hencky model shows no residual stresses even in the case where only two increments are used per cycle.

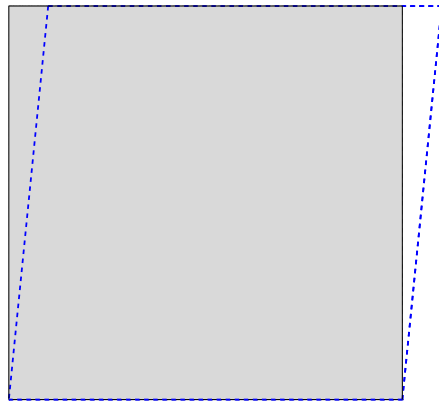


Figure 30: Reference (shaded area) and final (dashed line) configurations of a specimen subjected to simple shear oscillations.

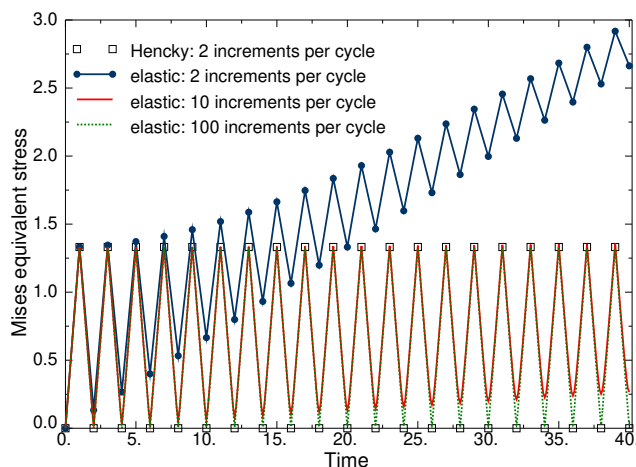


Figure 31: Mises equivalent stress as a function of time.

For more information, see *Hyperelastic Behavior of Rubberlike Materials*.

2024 FD01

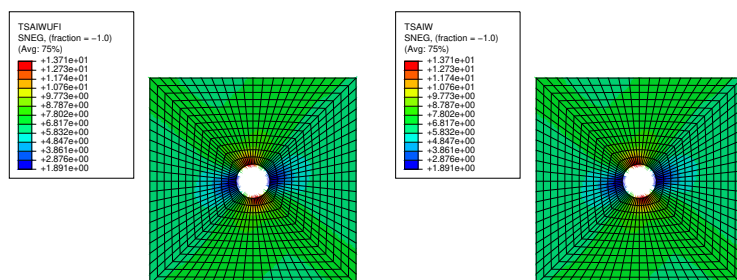
Generalized Damage and Failure for Anisotropic Materials

You can now specify new failure criteria for fully anisotropic materials to evaluate the strength of the material based on stress or strain results. You can also use the same failure criteria at the pseudo-grain level for multiscale materials.

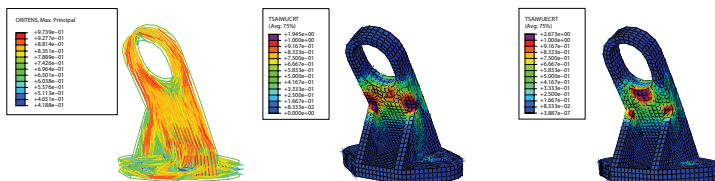
Benefits: The new stress/strain-based failure criteria are available with any material type and all stress/displacement elements, which gives you more options to evaluate the strength of materials.

Abaqus now provides a collection of stress/strain-based generalized damage initiation criteria as indicators of material failure. These general stress/strain-based damage initiation criteria are based on Tsai-Wu theory or maximum stress/strain theory. Abaqus does not support damage evolution laws for these damage criteria.

The figure below shows the failure index evaluated using the Tsai-Wu failure theory in an open hole tension test. The coupon plate is modeled with composite solid elements with $[45/0/45/90]_s$ layups.



The following figure shows the failure criteria evaluated using the Tsai-Wu stress and strain theory at the pseudo-grain level in an injection-molded plastic part. The maximum principal direction of the fiber orientation tensor is also shown below.



For more information, see *Generalized Damage and Failure for Anisotropic Materials*.

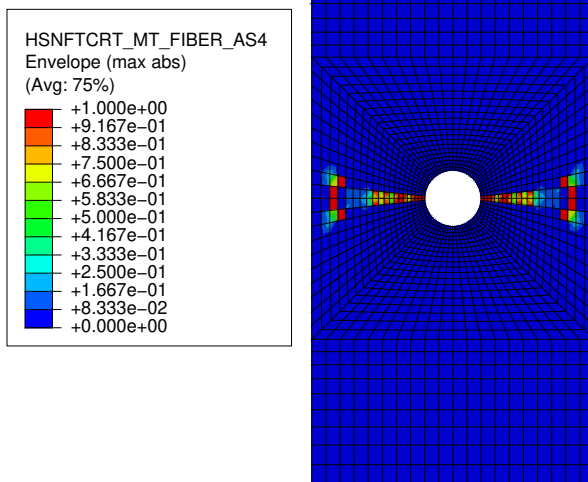
Damage and Failure for Fiber-Reinforced Composites Using Multiscale Modeling

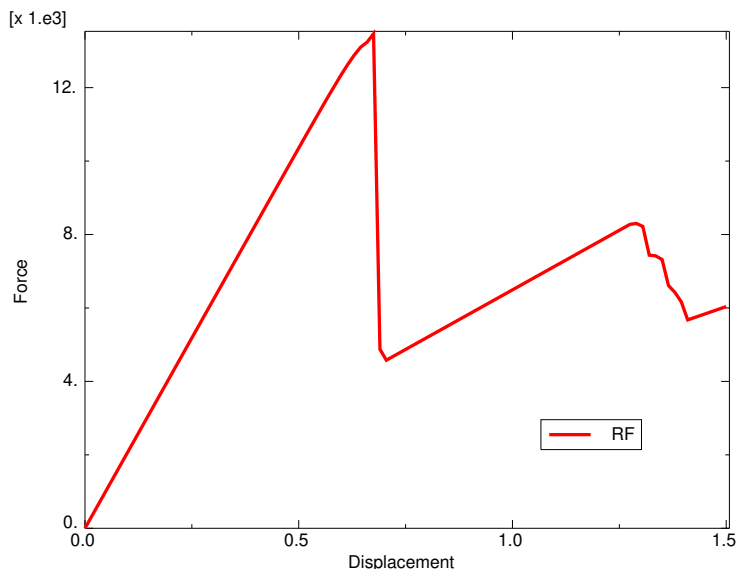
You can now specify new failure criteria for fiber-reinforced composite materials to evaluate the strength of the material based on stress invariants at the micro-level. You can also specify damage evolution laws for these damage criteria.

Benefits: The new damage initiation criteria allow you to evaluate damage at the composite constituent level using multiscale modeling, which simplifies damage modeling.

Abaqus/Standard offers capabilities for predicting the onset of damage and for modeling progressive damage and failure at the constituent level in fiber-reinforced composites, which include unidirectional and woven fiber-reinforced composite materials. You can use the criteria with both 2D and 3D solid and shell stress/displacement elements. You can specify only linear responses in the constituents. You can also use the new damage initiation criteria in 3D elements; in earlier releases, these criteria were available for plane stress elements only.

The figures below show the results of an open hole tension test modeled with composite solid elements with $[45/0/45/90]_s$ layups. The top figure shows the fiber damage initiation criteria, and the bottom figure shows the reaction force result versus the displacement.





For more information, see *Damage and Failure for Fiber-Reinforced Composites Using Multiscale Modeling*.

Field Expansion in Pore Fluid in a Porous Medium

You can now use field expansion with the pore fluid phase in a porous medium.

Benefits: The ability to define field expansion in the solid and the fluid phases in a porous medium independently provides a convenient way to introduce expansion of the material due to physical sources other than temperature, without having to model such sources explicitly.

In earlier releases, you could only define field expansion for the solid grains that make up the skeleton in a porous medium. Now, you can define field expansion in the different phases in a porous medium; that is, the solid grains and the pore fluid, independently. You can define only isotropic field expansion behavior for the pore fluid.

In addition, you can also output the strains due to both field expansion and thermal expansion in the pore fluid.

For more information, see *Thermal Expansion* and *Field Expansion*.

Band-Limited Damping in Abaqus/Standard

You can now specify band-limited damping in Abaqus/Standard.

Benefits: The band-limited damping allows you to specify a damping ratio that remains approximately constant within a frequency range specified.

In many solid materials subjected to oscillatory loads, the energy dissipated in a cycle is controlled by internal friction and is insensitive to the frequency of the oscillation. This behavior cannot be captured by the stiffness proportional (beta) or mass proportional (alpha) damping options previously available in Abaqus/Standard, but can be modeled using the band-limited damping option. For this model, you specify the damping ratio, which can be defined as a function of temperature variables, field variables, or both. In addition, you provide the low and high frequency cutoff values to specify the frequency range, and Abaqus maintains approximately constant

value of the damping ratio (and energy dissipated per cycle) within this range. In Abaqus/Standard, the effect of band-limited damping is considered only with the implicit dynamic procedure.

Figure 3 shows a bar subjected to harmonic excitations. The bar is fixed at the left end, and a displacement boundary condition is applied at the other end with different forced frequencies. The bar is modeled using a linearly elastic material in combination with band-limited damping. A constant damping ratio of 0.02 is specified, the low angular frequency is 1, and the high angular frequency is 10. *Figure 4* shows the energy dissipated as a function of angular frequency, which remains approximately constant within the frequency range specified and decreases rapidly outside of the range. The damping ratio, obtained by scaling the energy dissipated in a cycle by the maximum value of the strain energy, is shown in *Figure 5* and follows a similar pattern. As expected, Abaqus achieves a damping ratio approximately equal to 0.02 within the specified range of frequencies.



Figure 32: Bar subjected to harmonic excitations.

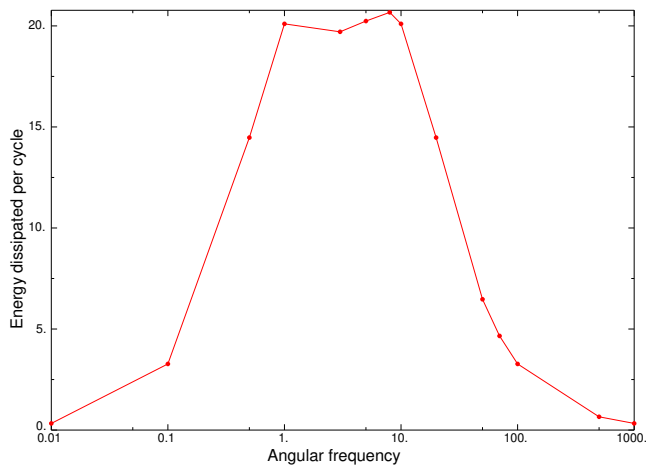


Figure 33: Energy dissipated per cycle as a function of angular frequency.

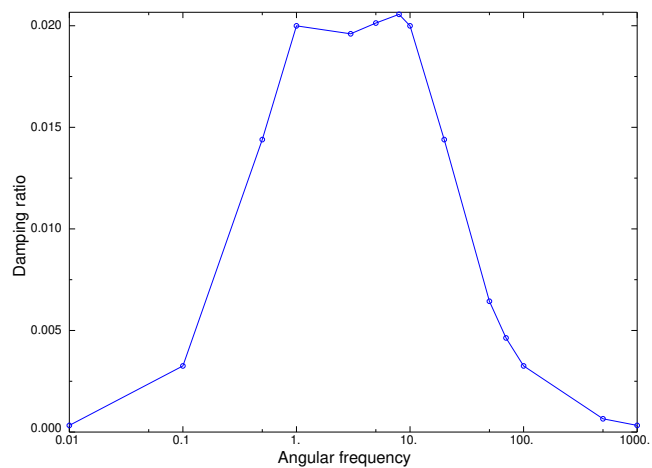


Figure 34: Damping ratio as a function of angular frequency.

For more information, see *Material Damping*.

Output

This page describes recent changes in Abaqus Output.

2024 FD03

New Output Variables

New variables are available.

Benefits: The new output variables expand the capabilities of Abaqus/Standard and Abaqus/Explicit.

Element Integration Point Variables

Output Variable	Field	History	.fil	.dat	Product
SOC_ <i>i</i> State of charge in particle <i>i</i> (1 <i>i</i> 3).	yes	yes	no	no	Abaqus/Standard

Element Section Variables

Output Variable	Field	History	.fil	.dat	Product
UVARPT Element user-defined output variables.	yes	yes	no	no	Abaqus/Standard
UVARPT _{<i>n</i>} Element user-defined output variable <i>n</i> .	yes	yes	no	no	Abaqus/Standard

Whole Element Variables

Output Variable	Field	History	.fil	.dat	Product
UVARE Element user-defined output variables.	yes	yes	no	no	Abaqus/Standard
UVARE _{<i>n</i>} Element user-defined output variable <i>n</i> .	yes	yes	no	no	Abaqus/Standard

Whole and Partial Model Variables

Output Variable	Field	History	.fil	.dat	Product
SOC Current state of charge in the	no	yes	no	no	Abaqus/Standard

Output Variable	Field	History	.fil	.dat	Product
specified region of the model.					

For more information, see *Element Integration Point Variables*, *Whole Element Variables*, and *Whole and Partial Model Variables*.

2024 FD02

New Output Variables

New variables are available.

Benefits: The new output variables expand the capabilities of Abaqus/Standard and Abaqus/Explicit.

Element Integration Point Variables

Output Variable	Field	History	.fil	.dat	Product
PORVAVG Equivalent pore pressure, computed as a volume fraction weighted average of all materials in the element.	yes	no	no		Abaqus/Explicit
SDEFRES Deformation resistance in Anand's creep model.	yes	yes	no	no	Abaqus/Standard
TG Glass transition temperature.	yes	yes	no	no	Abaqus/Standard
TGTDIFF The difference between the glass transition temperature and the temperature.	yes	yes	no	no	Abaqus/Standard

Element Section Variables

Output Variable	Field	History	.fil	.dat	Product
SK	yes	yes	yes	yes	Abaqus/Standard

Output Variable	Field	History	.fil	.dat	Product
Section curvature change and twist components.					

For more information, see *Element Integration Point Variables*, *Element Section Variables*, and *Element Integration Point Variables*.

2024 FD01

New Output Variables

New variables are available.

Benefits: The new output variables expand the capabilities of Abaqus/Standard and Abaqus/Explicit.

Element Integration Point Variables

Output Variable	Field	History	.fil	.dat	Product
MSTRESSCRT	yes	yes	no	no	Abaqus/Standard
Maximum stress-based damage initiation criterion.	no	yes	no		Abaqus/Explicit
MSTRAINCRT	yes	yes	no	no	Abaqus/Standard
Maximum strain-based damage initiation criterion.	no	yes	no		Abaqus/Explicit
TSAIWUCRT	yes	yes	no	no	Abaqus/Standard
Tsai-wu stress-based damage initiation criterion.	no	yes	no		Abaqus/Explicit
TSAIWUECRT	yes	yes	no	no	Abaqus/Standard
Tsai-wu strain-based damage initiation criterion.	no	yes	no		Abaqus/Explicit
DMIFI	yes	yes	no	no	Abaqus/Standard
All active components of the damage initiation failure indices.	yes	yes	no		Abaqus/Explicit
MSTRESSFI	yes	yes	no	no	Abaqus/Standard
Maximum stress-based damage	no	yes	no		Abaqus/Explicit

Output Variable	Field	History	.fil	.dat	Product
initiation failure index.					
MSTRAINFI	yes	yes	no	no	Abaqus/Standard
Maximum strain-based damage initiation failure index.	no	yes	no		Abaqus/Explicit
TSAIWUFI	yes	yes	no	no	Abaqus/Standard
Tsai-Wu stress-based damage initiation failure index.	no	yes	no		Abaqus/Explicit
TSAIWUEFI	yes	yes	no	no	Abaqus/Standard
Tsai-Wu strain-based damage initiation failure index.	no	yes	no		Abaqus/Explicit
TSINVMTCRT	yes	yes	no	yes	Abaqus/Standard
Transversely isotropic stress invariant-based matrix tensile damage initiation criterion.					
TSINVMCCRT	yes	yes	no	yes	Abaqus/Standard
Transversely isotropic stress invariant-based matrix compression damage initiation criterion.					
THEFL	yes	yes	no	no	Abaqus/Standard
Thermal strain in the pore fluid in a porous medium.					
FVE	yes	yes	no	no	Abaqus/Standard
All tensor components of all field expansion strain tensors.					
FVEFL	yes	yes	no	no	Abaqus/Standard

Output Variable	Field	History	.fil	.dat	Product
All field expansion strains in the pore fluid in a porous medium.					
FVE n All tensor components for the field expansion strain tensor due to field variable number n .	yes	yes	no	no	Abaqus/Standard
FVEFL n Field expansion strain in the pore fluid in a porous medium due to field variable number n .	yes	yes	no	no	Abaqus/Standard

Element Face Variables

Output Variable	Field	History	.fil	.dat	Product
CWEAR Accumulated scalar nodal contact wear distance.	yes	yes	no	no	Abaqus/Standard

Surface Variables

Output Variable	Field	History	.fil	.dat	Product
AMBIENTTEMP Reference ambient temperature on element faces from prescribed boundary radiation.	yes	no	no	no	Abaqus/Standard

For more information, see *Element Integration Point Variables* and *Element Integration Point Variables*.

2024 GA

New Output Variables

New variables are available.

Benefits: The new output variables expand the capabilities of Abaqus/Standard and Abaqus/Explicit.

Nodal Variables

Output Variable	Field	History	.fil	.dat	Product
STIFN Local normal stiffness.	yes	no	no	no	Abaqus/Standard
VN Complex-valued surface normal velocity.	yes	yes	no	no	Abaqus/Standard—available only for steady-state (frequency domain) dynamic analyses (modal and direct)
VNSQ Real-valued surface normal velocity squared.	yes	yes	no	no	Abaqus/Standard—available only for steady-state (frequency domain) dynamic analyses (modal and direct)
AVNSQ Area-weighted surface normal velocity squared or acoustic power normalized by the acoustic impedance of the surrounding fluid.	yes	yes	no	no	Abaqus/Standard—available only for steady-state (frequency domain) dynamic analyses (modal and direct)
FEXT All components of external point loads from a co-simulation or external field definition.	yes	yes	no		Abaqus/Explicit
MEXT All components of external point moments from a co-simulation or external field definition.	yes	yes	no		Abaqus/Explicit

Structural Optimization Variables

Output Variable	Field	History	.fil	.dat	Product
DISP_NORMAL_VAL The value of the bead optimization		no	no	no	Abaqus/Standard—output automatically during shape optimization

Output Variable	Field	History	.fil	.dat	Product
displacement along the node normal vector.					

For more information, see *Abaqus/Standard Nodal Variables*, *Structural Optimization Variables*, and *Abaqus/Explicit Nodal Variables*.

Prescribed Conditions

This page describes recent changes in Abaqus Prescribed Conditions.

2024 FD01

Modified Interface to Define Time Variation of Ambient Temperature for Thermal Radiation

For prescribed thermal radiation boundary conditions, the new AMBIENT AMPLITUDE replaces the AMPLITUDE parameter available in earlier versions.

Benefits: The new parameter name better describes the capability, which enhances the clarity of the user interface. The new AMBIENT AMPLITUDE parameter behaves identically to the AMPLITUDE parameter available in earlier releases. Abaqus will continue to support the AMPLITUDE parameter for some time; however, you are encouraged to replace it with the new parameter in all your simulations.

For more information, see **RADIATE*, **CRADIATE*, **SRADIATE*, and *Thermal Loads*.

Time Variation of Radiation Flux

For prescribed thermal radiation boundary conditions, you can use the new RADIATION AMPLITUDE parameter to define the time variation of the prescribed radiation flux directly through an amplitude definition.

Benefits: This capability provides additional modeling flexibility in some situations.

For more information, see **RADIATE*, **CRADIATE*, **SRADIATE*, and *Thermal Loads*.

Modified Interface to Define Time Variation of Sink Temperature for Convection Boundary Conditions

For prescribed convective (film) boundary conditions in a thermal analysis, the new SINK AMPLITUDE parameter replaces the AMPLITUDE parameter available in earlier versions.

Benefits: The new parameter name better describes the capability, which enhances the clarity of the user interface. The new SINK AMPLITUDE parameter behaves identically to the AMPLITUDE parameter available in earlier releases. Abaqus will continue to support the AMPLITUDE parameter for some time; however, you are encouraged to replace it with the new parameter in all your simulations.

For more information, see **FILM*, **CFILM*, **SFILM*, and *Thermal Loads*.

Algorithmic Improvements to Convection and Radiation Boundary Flux Calculations

The flux calculations associated with prescribed convection and/or radiation boundary conditions in Abaqus/Standard now use an improved algorithm.

Benefits: The improved algorithms ensure continuity of convective/radiative fluxes across step boundaries, resulting in smooth temperature variations, and lead to better overall convergence characteristics for certain types of multi-step steady-state thermal analyses.

By default, in the first step of a steady-state thermal analysis earlier versions of Abaqus/Standard assumed that the sink temperature was ramped up from zero to its specified value over the step time period. In the presence of prescribed nonzero initial temperatures, this assumption led to discontinuities in the convective/radiative fluxes, as well as the temperature fields. In addition, in a multi-step analysis, the default amplitude behavior associated with film/radiation thermal loads could result in nonsmooth temperature and heat fluxes across step boundaries. Such discontinuities could also lead to convergence difficulties in nonlinear problems. The new, improved algorithm removes the default modulation of the sink temperature by the step amplitude, ensuring continuity of fluxes, smooth variations of temperature, and better overall convergence. For more information, see the Knowledge Base item QA00000307343 in the Dassault Systèmes Knowledge Base.

Figure 1 shows a one-element test with prescribed temperatures at the bottom nodes, a prescribed heat flux along the bottom surface, and a prescribed convective (film) boundary condition on the top surface. There are two steady-state heat transfer steps in the analysis, each with a time period of 1.0 (for a total time period of 2.0).

Figure 2 and *Figure 3* show the temperature versus time and convective heat flux versus time. In the old formulation, the convective heat flux experienced a discontinuity at the transition between the two steps. The improved formulation leads to a smoother behavior of the convective heat flux over the second step.

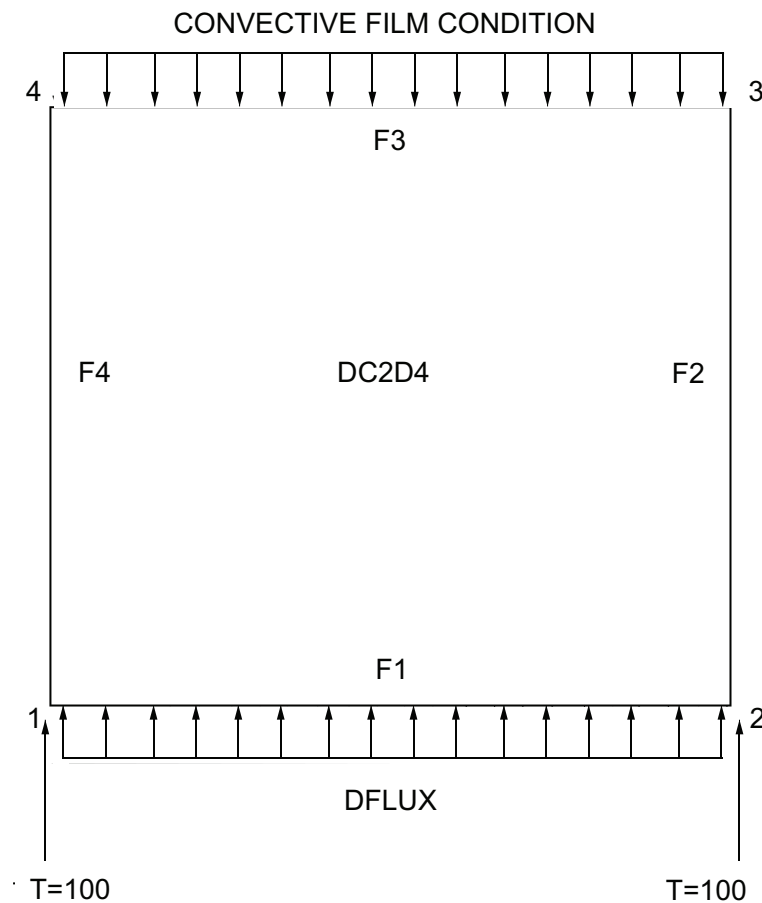


Figure 35: A steady-state thermal analysis with prescribed distributed heat flux on the bottom surface and convective boundary conditions on the top surface.

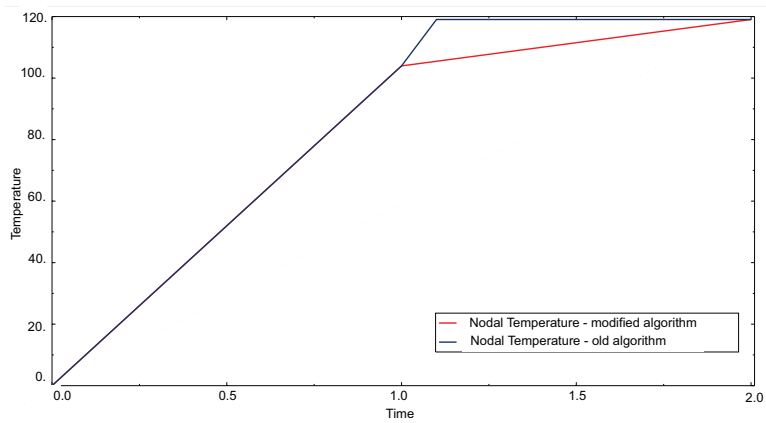


Figure 36: Contrasting the temperatures across step transitions using the old and the improved (modified) algorithms.

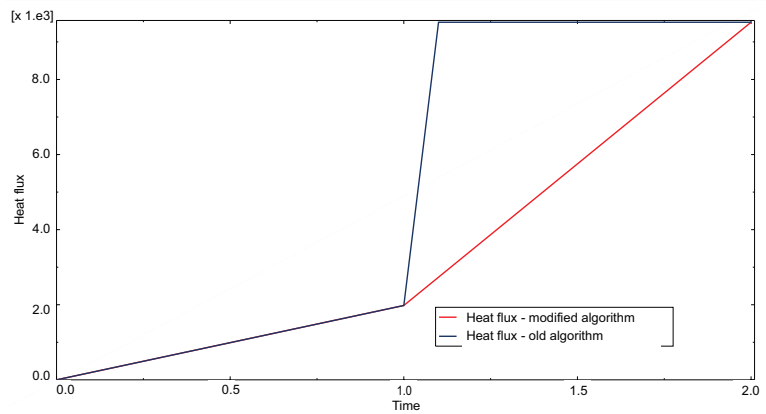


Figure 37: Contrasting the convective heat fluxes across step transitions using the old and the improved (modified) algorithms.

For more information, see **FILM*, **CFILM*, **SFILM*, **RADIATE*, **CRADIATE*, **SRADIATE*, and *Thermal Loads*.

Scripting

This page describes recent changes in Abaqus Scripting.

2024 GA

Abaqus Python Upgrade to Version 3.10.5

Abaqus Python is updated to Version 3.10.5.

Benefits: Customers and partners can leverage the latest functionality provided in Python 3.

AbaqusPython has been updated from Python version 2.7.15 to Python version 3.10.5.

Python 3 introduces a number of changes that break backward-compatibility with existing Python 2 code. As a result, scripts originally written using Python 2 syntax might not run without modification in Python 3. However, in most cases it is possible to modify your scripts so that they will run in both Python 2.7 and Python 3.

The following Abaqus functionality is impacted:

- Abaqus/CAE and Abaqus/Viewer scripting
- Abaqus/CAE and Abaqus/Viewer plugins
- The `abaqus_v6.env` environment file (see *Using the Abaqus environment files*)
- Custom applications (see *Creating a customized application*)
- Standalone Abaqus Python scripts; for example, scripts run with `abaqus python myscript.py` (see *Python Execution*)
- `*PARAMETER` (see *Parametric Modeling*)
- Parametric studies—`.psf` files (see *Parametric Studies*)
- Batch scheduling scripts (Dassault Systèmes Knowledge Base, <https://support.3ds.com/knowledge-base>)

A conversion utility is provided to assist with converting Python 2 syntax to Python 3. While it will automatically convert many common syntax changes, not all Python 2 syntax can be converted automatically; therefore, some manual conversion might be required to ensure that Python scripts run properly with the new version. You can invoke the conversion utility from the command line using the command

```
abaqus python -m abqPy2to3 <script/directory>
```

The utility writes a log file and saves the original versions of the files on which it operates. If you provide a directory to the utility, it recursively traverses through that directory and subdirectories looking for `*.py` files. You can include `-future False` on the command line if you are not worried about maintaining Python 2.7 forward compatibility. You can also access the conversion utility as part of the script upgrade plug-in utility in Abaqus/CAE (available from **Plug-Ins > Abaqus > Upgrade Scripts**). It is run if you upgrade scripts to 2024.

Additional information related to converting Python code for Abaqus is available in the SIMULIA Community at go.3ds.com/AbaqusPython.

Scripting Reference

This page describes recent changes in Abaqus Scripting Reference.

2025 GA

Python Interactions Commands: FluidExchangeActivation Object

The FluidExchangeActivation object is now available in the Interactions profile commands.

Benefits: The feature extends the capabilities of the Interactions commands.

For more information, see *FluidExchangeActivation object*.

Python Interactions Commands: FluidExchangeActivationState Object

The FluidExchangeActivationState object is now available in the Interactions profile commands.

Benefits: The feature extends the capabilities of the Interactions commands.

For more information, see *FluidExchangeActivationState object*.

Python Interactions Commands: FluidInflator Object

The required arguments inflationTimeAmplitude and massFlowAmplitude are no longer available in the FluidInflator object.

Benefits: The functionality was extended, with these arguments appropriately available in the FluidInflatorActivation object instead.

For more information, see *FluidInflator object*.

Python Interactions Commands: FluidInflatorActivation Object

The FluidInflatorActivation object is now available in the Interactions profile commands.

Benefits: The feature extends the capabilities of the Interactions commands.

For more information, see *FluidInflatorActivation object*.

Python Interactions Commands: FluidInflatorActivationState Object

The FluidInflatorActivationState object is now available in the Interactions commands.

Benefits: The feature extends the capabilities of the Interactions commands.

For more information, see *FluidInflatorActivationState object*.

Python Interactions Commands: WearProperty Object

The WearProperty object is now available in the Interactions commands.

Benefits: The feature extends the capabilities of the Interactions commands.

For more information, see *WearProperty object*.

Python Interactions Commands: WearSurfacePropertyAssignment Object

The WearSurfacePropertyAssignment object is now available in the Interactions commands.

Benefits: The feature extends the capabilities of the Interactions commands.

For more information, see *WearSurfacePropertyAssignment object*.

Python Load and Load Case Commands: RotationalBodyForce Object

The new optional argument rotorDynamicloads is available in the RotationalBodyForce object.

Benefits: The feature extends the capabilities of the Load and Load Case commands.

For more information, see *RotationalBodyForce object*.

Python Path and Probe Commands: FreeBody Objects

The method getFreeBodyData() is now available in the FreeBody objects.

Benefits: The feature extends the capabilities of the Path and Probe commands.

For more information, see *FreeBody object*.

2024 FD03

New Method getMemberFromAll for Node and Element Sequences

There is a new Python API on node and element sequences to retrieve a member value for each item in the sequence. This applies to MeshNodeArray, MeshElementArray, MeshFaceArray, ODBMeshNodeArray, and ODBMeshElementArray.

Benefits: Retrieving a member from a large sequence (for example, the node label for every node) is much faster using this method than iterating over the sequence.

For more information, see *MeshNodeArray object*, *MeshElementArray object*, *MeshFaceArray object*, *OdbMeshNode object*, and *OdbMeshElement object*.

2024 GA

Python 3

Abaqus Python and Abaqus/CAE are now based on Python 3.10 instead of Python 2.7.

Benefits: This update brings Abaqus Python into conformance with a version of Python that is supported in the open-source community.

For more information, see <https://go.3ds.com/AbaqusPython>.

Python Beam Section Profile Commands: ChannelProfile Object

The ChannelProfile object is now available in the Beam Section profile commands.

Benefits: The feature extends the capabilities of the Beam Section profile commands.

For more information, see *ChannelProfile object*.

Python Beam Section Profile Commands: HatProfile Object

The HatProfile object is now available in the Beam Section profile commands.

Benefits: The feature extends the capabilities of the Beam Section profile commands.

For more information, see *HatProfile object*.

Python Constraint Commands: Coupling Object

The Coupling object is now available in the Constraint commands.

Benefits: The feature extends the capabilities of the Constraint commands.

For more information, see *Coupling object*.

Python Engineering Feature Commands: DiscreteFastener Object

The optional argument *rotationalCoupling* is now available in the DiscreteFastener object.

Benefits: The feature extends the capabilities of the Engineering Feature commands.

For more information, see *DiscreteFastener object*.

Python Interaction Commands: ContactMassScalingExp Object

The ContactMassScalingExp object is now available in the Interaction commands.

Benefits: The feature extends the capabilities of the Interaction commands.

For more information, see *ContactMassScalingExp object*.

Python Job Commands: OptimizationProcess Object

The optional argument *licenseType* is now available in the OptimizationProcess object.

Benefits: The feature extends the capabilities of the Job commands.

For more information, see *ContactMassScalingExp object*.

Python Odb Commands: OdbAssembly Object

The method *getMassProperties* is now available in the OdbAssembly object.

Benefits: The feature extends the capabilities of the Odb commands.

For more information, see *OdbAssembly object*.

Python Optimization Commands: SingleTermDesignResponse Object

The optional argument `stepOperation` is no longer available in the `SingleTermDesignResponse` object.

Benefits: The functionality was removed from the Optimization commands: `SingleTermDesignResponse` object.

For more information, see *SingleTermDesignResponse object*.

Python Section Commands: BeamSection Object

The optional argument `beamSectionOffset` is now available in the `BeamSection` object.

Benefits: The feature extends the capabilities of the Section commands.

For more information, see *BeamSection object*.

Python Step Commands (Step): StaticLinearPerturbationStep Object

The optional arguments `solutionTechnique`, `gapDistance`, and `scaleFactor` are now available in the `StaticLinearPerturbationStep` object.

Benefits: The feature extends the capabilities of the Step commands (step).

For more information, see *StaticLinearPerturbationStep object*.

Python XY Commands: Session Object

The optional argument `useMaxStressValue` is now available in the `Session` object.

Benefits: The feature extends the capabilities of the XY commands.

For more information, see *Session object*.

Theory

This page describes recent changes in Abaqus Theory.

2024 FD01

Coupled Acoustic-Structural Medium Analysis

The theory behind the use of secondary output quantities in acoustic analysis is discussed.

Benefits: The discussion expands the definition of the theories used in Abaqus.

For more information, see *Coupled acoustic-structural medium analysis*.

User Subroutines

This page describes recent changes in Abaqus User Subroutines.

2024 FD03

UVAREL (new)

You can use the new user subroutine UVAREL to define output variables at the element and integration point level in Abaqus/Standard.

Benefits: The user subroutine makes it easier for you to compute, create, and elaborate user-defined element output variables and provides you the flexibility to implement a broad variety of data consolidation schemes that can reduce the amount of results that must be stored in the output database.

User subroutine UVAREL gives you a method to define variables at the element level to reduce the amount of output data if the results at every material point are not needed. The reduction of output data can be significant, especially for composite elements with many layers.

The user-defined element output variables that are defined in user subroutine UVAREL usually depend on other Abaqus output variables such as stresses, strains, and section forces. You can access these output quantities by calling the new utility routines `getElMPtVar` and `getSecVar`.

You can invoke the following new utility routines from user subroutine UVAREL:

- `getElMPtVar` returns material point information
- `getSecVar` returns integration point information
- `getUVarElOrientationName` returns the name of the element orientation
- `getUVarElMaterialName` returns the name of the material

For more information, see *UVAREL, *ELEMENT USER OUTPUT VARIABLES, Obtaining Material Point Information from User Subroutine UVAREL, Obtaining the Element Orientation Name from User Subroutine UVAREL, Obtaining the Material Name from User Subroutine UVAREL, Obtaining Element Section Information from User Subroutine UVAREL, and UVAREL*.

User Subroutine and Utilities Assistance Consolidated in this Guide

Conceptual information about user subroutines and utilities available in Abaqus is now consolidated in this guide.

Benefits: Moving the content from the *Abaqus Analysis Guide* makes it easier for you to access the information as you review the reference topics in this guide.

For more information, see *About User Subroutines and Utilities, About Table Collections, Parameter Tables, and Property Tables, and About Event Series*.

2024 FD02

VDLOAD

You can now access displacement and acceleration information in user subroutine `VDLOAD`.

Benefits: Accessing the displacements and accelerations information in the user subroutine is useful when the distributed loads are a function of the displacements and accelerations.

The array `velocity` available in earlier releases in the `VDLOAD` argument list is now named `vua`. It is also extended to include velocities, displacements, and accelerations, giving you the option to define distributed loads as function of these quantities with user subroutine `VDLOAD`. This change does not affect existing `VDLOAD` implementations based on the interface available in earlier releases.

For more information, see *VDLOAD*.

2024 FD01

UEXPAN

You can now define field expansion strains and, in a porous medium, pore fluid field expansion strains directly in user subroutine `UEXPAN`.

Benefits: Defining the values in the user subroutine is particularly useful when the field expansion strains are complicated functions of temperature, field variables, or state variables.

The argument list of user subroutine `UEXPAN` has been enhanced to provide additional information that you can use to discern if the call corresponds to a thermal or a field expansion definition, the field number and, in the case of a porous medium, if the call corresponds to the expansion definition for the pore fluid. This change does not affect existing `UEXPAN` implementations based on the interface available in earlier releases.

For more information, see *UEXPAN* and *UEXPAN*.

Verification

This page describes recent changes in Abaqus Verification.

2025 GA

Contact Wear Modeling

New tests verify step cycling controls for general contact.

Benefits: The tests verify the capability in Abaqus/Explicit.

For more information, see *Contact wear modeling*.

2024 FD04

Contact Wear Modeling

New tests verify the contact wear capability for general contact.

Benefits: The tests verify the capability in Abaqus/Explicit.

For more information, see *Contact wear modeling*.

2024 FD03

UVAREL

New tests verify variables that are passed into user subroutine UVAREL and integration point quantities that are read by utility subroutine `getELMPtVar`.

Benefits: The tests verify the capability in Abaqus/Standard.

For more information, see *UVAREL*.

Piezoresistive Effect

New tests model one-dimensional electrical current flow using axisymmetric and three-dimensional elements.

Benefits: The tests verify the capability in Abaqus/Standard.

For more information, see *Piezoresistivity properties: Piezoresistive effect*.

Bending Stabilization

New tests verify the bending stabilization capability.

Benefits: The tests verify the capability in Abaqus/Standard.

For more information, see *Bending stabilization for full-integration linear elements* and *Using Section Controls for Bending Stabilization*.

User-Defined Element Output Variables

New tests verify the definition and output of user-defined element variables.

Benefits: The tests verify the capability in Abaqus/Standard.

For more information, see *UVAREL* and **ELEMENT USER OUTPUT VARIABLES*.

Thermal Expansion

New tests verify the tangent and secant thermal expansion capability.

Benefits: The tests verify the capability in Abaqus/Standard.

For more information, see *Thermal Expansion*.

Stretching of Solid Body Using SPH Element Conversion

New tests verify the accuracy of Lagrangian element (C3D8R) to SPH particle (PC3D) conversion for a solid body subjected to uniaxial stretching.

Benefits: The tests verify the capability in Abaqus/Explicit.

For more information, see *Smoothed particle hydrodynamic analysis*.

Progressive Damage and Failure in Fiber-Reinforced Materials

A new Python test models progressive damage and failure in fiber-reinforced materials with multiscale modeling.

Benefits: The test verifies the capability in Abaqus/Standard.

For more information, see *Progressive damage and failure in fiber-reinforced materials*.

2024 FD02

Undrained Pore Fluid Flow and Stress Analysis

New tests verify undrained pore fluid flow and stress analysis.

Benefits: The tests verify the capability in Abaqus/Explicit.

A unit cube of geometry with distorted meshes is used to test three-dimensional solid elements, and a simple rectangular geometry is used to test plane strain elements as well as axisymmetric solid elements. The medium is subject to prescribed displacement boundary conditions, and the numerical results are compared with a reference solution.

For more information, see *Undrained pore fluid flow and stress analysis*.

2024 FD01

Progressive Damage and Failure in Fiber-Reinforced Materials

New tests contain basic cases to model progressive damage and failure in fiber-reinforced materials with multiscale modeling.

Benefits: The tests verify the capability in Abaqus/Standard.

For more information, see *Progressive damage and failure in fiber-reinforced materials*.

Generalized Damage and Failure for Anisotropic Materials

New tests contain basic cases with various damage initiation criteria for anisotropic materials.

Benefits: The tests verify the capability in Abaqus/Standard and Abaqus/Explicit.

For more information, see *Generalized damage and failure for anisotropic materials*.

Field Expansion

New tests verify the field expansion capability for both solid and pore fluid phases in a porous medium.

Benefits: The tests verify the capability in Abaqus/Standard.

The solid grains and the pore fluid in a porous medium are subjected to thermal expansion and multiple field expansions. Both built-in capabilities and user subroutine *UEXPAN* are used.

For more information, see *Field expansion test, UEXPAN, and UEXPAN*.

Band-Limited Damping in Abaqus/Standard

New tests verify the use of band-limited damping with various element types and material models.

Benefits: The tests verify the capability in Abaqus/Standard.

For more information, see *Material Damping*.

Updated SLIPRING Contact Angle in Abaqus/Standard

New tests verify the use of the updated contact angle (based on the nodal coordinates) for the SLIPRING connection type in Abaqus/Standard.

Benefits: The tests verify the capability in Abaqus/Standard.

For more information, see *Frictional SLIPRING connectors*.

Contact Wear Modeling

New tests verify the contact wear capability for both general contact and contact pairs.

Benefits: The tests verify the capability in Abaqus/Standard.

For more information, see *Contact wear modeling*.

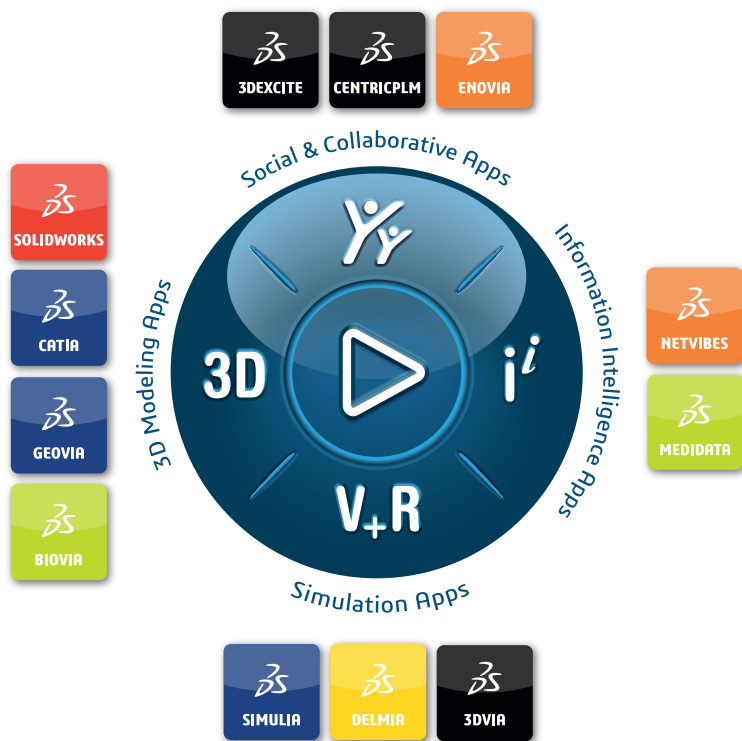
2024 GA

Tie Constraints

New tests verify that the configuration used for the formation of tie constraints in an Abaqus/Explicit import analysis depends on whether or not there are changes to the definitions of the surfaces to be tied.

Benefits: The tests verify the capability in Abaqus/Explicit.

For more information, see *Miscellaneous results transfer tests*.



Our 3DEXPERIENCE® platform powers our brand applications, serving 12 industries, and provides a rich portfolio of industry solution experiences.

Dassault Systèmes is a catalyst for human progress. We provide business and people with collaborative virtual environments to imagine sustainable innovations. By creating virtual twin experiences of the real world with our 3DEXPERIENCE platform and applications, our customers can redefine the creation, production and life-cycle-management processes of their offer and thus have a meaningful impact to make the world more sustainable. The beauty of the Experience Economy is that it is a human-centered economy for the benefit of all – consumers, patients and citizens.

Dassault Systèmes brings value to more than 300,000 customers of all sizes, in all industries, in more than 150 countries. For more information, visit www.3ds.com.

Europe/Middle East/Africa
Dassault Systèmes
10, rue Marcel Dassault
CS 40501
78946 Vélizy-Villacoublay Cedex
France

Asia-Pacific
Dassault Systèmes
17F, Foxconn Building,
No. 1366, Lujiazui Ring Road
Pilot Free Trade Zone, Shanghai 200120
China

Americas
Dassault Systèmes
175 Wyman Street
Waltham, Massachusetts
02451-1223
USA

